

# Satisfiability Modulo Exponential Integer Arithmetic

Florian Frohn · Jürgen Giesl

the date of receipt and acceptance should be inserted later

**Abstract** SMT solvers use sophisticated techniques for polynomial (linear or non-linear) integer arithmetic. In contrast, non-polynomial integer arithmetic has mostly been neglected so far. However, in the context of program verification, polynomials are often insufficient to capture the behavior of the analyzed system without resorting to approximations. In the last years, *incremental linearization* has been applied successfully to satisfiability modulo real arithmetic with transcendental functions. We adapt this approach to an extension of polynomial integer arithmetic with exponential functions. Here, the key challenge is to compute suitable *lemmas* that eliminate the current model from the search space if it violates the semantics of exponentiation.

We implemented our approach and evaluate it empirically on several sets of benchmarks from different domains: Most of them stem from program verification, and one set stems from recurrence solving. On all sets, our approach is highly effective in practice.

## 1 Introduction

Traditionally, automated reasoning techniques for integers focus on polynomial arithmetic. This is not only true in the context of SMT, but also for program verification techniques, since the latter often search for polynomial invariants that imply the desired properties. As invariants are over-approximations, they are well suited for proving “universal” properties like safety, termination, or upper bounds on the worst-case runtime that refer to all possible program runs. However, proving dual properties like unsafety, non-termination, or lower bounds requires under-approximations, so that invariants are of limited use here.

For lower bounds, an *infinite set* of witnesses is needed, as the runtime w.r.t. a finite set of (terminating) program runs is always bounded by a constant. Thus, to prove non-constant lower bounds, *symbolic under-approximations* are required,

i.e., formulas that describe an infinite subset of the reachable states. However, polynomial arithmetic is often insufficient to express such approximations. To see this, consider the program

```

x ← 1
y ← nondet(0, ∞)
while y > 0
|   x ← 3 · x
|   y ← y - 1

```

where `nondet`(0, ∞) returns a natural number non-deterministically. Here, the set of reachable states after the execution of the loop is characterized by the formula

$$\exists n \in \mathbb{N}. x = 3^n \wedge y = 0. \quad (1.1)$$

In recent work, *acceleration techniques* have successfully been used to deduce lower runtime bounds automatically [21, 22]. The purpose of such techniques is to characterize the transitive closure of a loop in a logic which is suitable for automated reasoning. While acceleration techniques can easily derive a formula like (1.1) from the code above, this is of limited use, as most<sup>1</sup> SMT solvers cannot handle terms of the form  $3^n$ . Besides lower bounds, acceleration has also successfully been used for proving non-termination [19, 22, 24] and (un)safety [4, 9, 10, 23, 33, 35], where its strength is finding long counterexamples that are challenging for other techniques.

Importantly, exponentiation is not just “yet another function” that can result from applying acceleration techniques. There are well-known, important classes of loops where polynomials and exponentiation *always* suffice to represent the values of the program variables after executing a loop [20, 31]. Thus, the lack of support for integer exponentiation in SMT solvers is a major obstacle for the further development of acceleration-based verification techniques.

In this work, we show how to lift standard SMT solvers to integer arithmetic with exponentiation, resulting in our novel tool `SwInE2` (SMT with Integer Exponentiation), built on top of the SMT solver `Z3` [39].

Our technique is inspired by *incremental linearization*, which has been applied successfully to *real* arithmetic with transcendental functions by Cimatti et al. [15], including the natural exponential function  $\exp_e(x) = e^x$ , where  $e$  is Euler’s number. In this setting, incremental linearization considers  $\exp_e$  as an uninterpreted function. If the resulting SMT problem is unsatisfiable, then so is the original problem. If it is satisfiable and the model that was found for  $\exp_e$  coincides with the semantics of exponentiation, then the original problem is satisfiable. Otherwise, *lemmas* about  $\exp_e$  that rule out the current model are added to the SMT problem, and then its satisfiability is checked again. The name “incremental linearization” is due to the fact that these lemmas only contain linear arithmetic.

The main challenge for adapting this approach to integer exponentiation is to generate suitable lemmas, see Sect. 3.2. Except for *monotonicity lemmas*, none of the lemmas of Cimatti et al. easily carry over to our setting. In contrast to Cimatti et al., we do not restrict ourselves to linear lemmas, but we also use non-linear, polynomial lemmas. This is due to the fact that we consider a binary version  $\lambda x, y. x^y$  of exponentiation, whereas Cimatti et al. fix the base to  $e$ . Thus, in our setting, one obtains *bilinear* lemmas that are linear w.r.t.  $x$  as well as  $y$ , but may

<sup>1</sup> cvc5 uses a dedicated solver for integer exponentiation with base 2.

contain multiplication between  $x$  and  $y$  (i.e., they may contain the subterm  $x \cdot y$ ). More precisely, bilinear lemmas arise from *bilinear interpolation*, which is a crucial ingredient of our approach, as it allows us to eliminate *any* model that violates the semantics of exponentiation (Thm. 3.26). Therefore, the name “incremental linearization” does not fit to our approach. Both approaches are instances of “counterexample-guided abstraction refinement” (CEGAR, [17]), but ours is more general than incremental linearization.

A preliminary shorter version of this paper appeared at IJCAR 2024 [26]. The current paper contains the following, yet unpublished contributions:

- Inspired by our work from [26], exponentiation has recently been added to the SMT-LIB [5] theory of integers,<sup>2</sup> but with slightly different semantics than the one that we used in [26]. Thus, we adapt our approach to the newly standardized semantics.
- We adapted our notion of “spurious counterexample” in such a way that our approach can often prove satisfiability with fewer lemmas.
- We present two new kinds of lemmas, called *prime lemmas* and *induction lemmas*.
- We present a new technique called *phasing*, which alternates between a **sat**-phase where the solver searches for a model within a restricted part of the search space, and an **unsat**-phase where the solver aims for proving unsatisfiability.
- While we did not give any proofs in [26], we now present proofs for all lemmas and theorems.
- We present SwInE2, a reimplementation of our approach, which directly builds on top of the API of the SMT solver Z3 [39]. In contrast, the implementation of our approach from [26] (called SwInE1 in the sequel) was built on top of SMT-Switch [37], a library that offers a unified interface for various SMT solvers. However, our evaluation in [26] showed that Z3 performs best in our setting, so that using its API directly allows for improving robustness.<sup>3</sup>
- We present a novel collection of challenging unsatisfiable benchmarks, which were obtained from verifying solutions for recurrence relations.
- We improve and extend our evaluation by including our novel benchmarks and the tools SwInE2, Z3, and cvc5. In order to use Z3 for integer exponentiation, we axiomatize the semantics of exponentiation via universally quantified assertions. For cvc5, we use the same approach, but in addition we also evaluate with its dedicated solver for exponentials with base 2.

To summarize, our contributions are as follows: Based on novel techniques for generating suitable lemmas, we develop a CEGAR approach for the newly standardized SMT theory EIA for integer arithmetic with exponentiation (Sect. 3). Next, we show how to improve our approach via phasing (Sect. 4). After discussing related work (Sect. 5), we present an experimental evaluation on several collections of EIA benchmarks that we synthesized from verification problems and from recurrence solving. Our experiments show that our approach is highly effective in practice (Sect. 6). We refer to App. A for all proofs.

<sup>2</sup> <https://smt-lib.org/theories-Ints.shtml>

<sup>3</sup> For example, SwInE1 uses the parser of SMT-Switch, which strictly adheres to the SMT-LIB standard. In contrast, Z3 (and thus SwInE2) can often also parse inputs that violate the SMT-LIB standard in a way that does not introduce ambiguity.

## 2 Preliminaries

We are working in the setting of *SMT-LIB logic* [5], a variant of many-sorted first-order logic with equality. We now introduce a reduced variant of SMT-LIB logic, where we only explain those concepts that are relevant for our work.

In SMT-LIB logic, there is a dedicated Boolean sort **Bool**, and hence formulas are just terms of sort **Bool**. Similarly, there is no distinction between predicates and functions, as predicates are simply functions with results of sort **Bool**.

So in SMT-LIB logic, a *signature*  $\Sigma = (\Sigma^S, \Sigma^F, \Sigma^R)$  consists of a set  $\Sigma^S$  of *sorts*, a set  $\Sigma^F$  of *function symbols*, and a *ranking function*  $\Sigma^R : \Sigma^F \rightarrow (\Sigma^S)^+$ . The meaning of  $\Sigma^R(f) = (s_1, \dots, s_k)$  is that  $f$  is a function which maps arguments of the sorts  $s_1, \dots, s_{k-1}$  to a result of sort  $s_k$ . We write  $f : s_1 \dots s_k$  instead of “ $f \in \Sigma^F$  and  $\Sigma^R(f) = (s_1, \dots, s_k)$ ” if  $\Sigma$  is clear from the context. We always allow to implicitly extend  $\Sigma$  with arbitrarily many constant function symbols (i.e., function symbols  $x$  where  $|\Sigma^R(x)| = 1$ ). Note that SMT-LIB logic only considers closed terms, i.e., terms without free variables, and we are only concerned with quantifier-free formulas, so in our setting, all formulas are ground. Therefore, we refer to these constant function symbols as *variables* to avoid confusion with other, predefined constant function symbols like **true**, 0, 1, ..., see below.

Every SMT-LIB signature is an extension of  $\Sigma_{\mathbf{Bool}}$  where  $\Sigma_{\mathbf{Bool}}^S = \{\mathbf{Bool}\}$  and  $\Sigma_{\mathbf{Bool}}^F$  consists of the following function symbols:

$$\mathbf{true}, \mathbf{false} : \mathbf{Bool} \quad \neg : \mathbf{Bool} \mathbf{Bool} \quad \wedge, \vee, \implies, \iff : \mathbf{Bool} \mathbf{Bool} \mathbf{Bool}$$

Note that SMT-LIB logic only considers well-sorted terms. A  $\Sigma$ -*structure*  $\mathbf{A}$  consists of a *universe*  $A = \bigcup_{s \in \Sigma^S} A_s$  and an *interpretation function* that maps each function symbol  $f : s_1 \dots s_k$  to a function  $\llbracket f \rrbracket^{\mathbf{A}} : A_{s_1} \times \dots \times A_{s_{k-1}} \rightarrow A_{s_k}$ . SMT-LIB logic only considers structures where  $A_{\mathbf{Bool}} = \{\mathbf{true}, \mathbf{false}\}$  and all function symbols from  $\Sigma_{\mathbf{Bool}}$  are interpreted as usual.

A  $\Sigma$ -*theory* is a class of  $\Sigma$ -structures. For example, consider the extension  $\Sigma_{\mathbf{Int}}$  of  $\Sigma_{\mathbf{Bool}}$  with the additional sort **Int** and the following function symbols:

$$\begin{aligned} c &: \mathbf{Int} && \text{for all } c \in \mathbb{N} \\ \mathbf{abs} &: \mathbf{Int} \mathbf{Int} \\ +, -, \cdot, \text{div}, \text{exp} &: \mathbf{Int} \mathbf{Int} \mathbf{Int} \\ <, \leq, >, \geq, =, \neq &: \mathbf{Int} \mathbf{Int} \mathbf{Bool} \\ \text{divisible}_c &: \mathbf{Int} \mathbf{Bool} && \text{for all } c \in \mathbb{N}_{>0} = \{n \in \mathbb{N} \mid n > 0\} \\ \text{ite} &: \mathbf{Bool} \mathbf{Int} \mathbf{Int} \end{aligned}$$

Then the  $\Sigma_{\mathbf{Int}}$ -theory *exponential integer arithmetic* (EIA)<sup>4</sup> contains all  $\Sigma_{\mathbf{Int}}$ -structures  $\mathbf{A}$  where  $A_{\mathbf{Int}} = \mathbb{Z}$ ,

<sup>4</sup> As we only consider quantifier-free formulas, we omit the prefix “QF\_” in theory names and write, e.g., EIA instead of QF\_EIA. Barrett et al. [5] call QF\_EIA an *SMT-LIB logic*, which restricts the (first-order) *theory* of integer arithmetic to the quantifier-free fragment. For simplicity, we do not distinguish between SMT-LIB logics and theories.

$$\begin{aligned} \llbracket \text{divisible}_c \rrbracket^{\mathbf{A}}(x) &= \begin{cases} \mathbf{true} & \text{if } x \bmod c = 0 \\ \mathbf{false} & \text{otherwise} \end{cases} \\ \llbracket \text{exp} \rrbracket^{\mathbf{A}}(x, y) &= \begin{cases} x^y & \text{if } x^y \in \mathbb{Z} \\ 0 & \text{otherwise} \end{cases} \\ \llbracket \text{ite} \rrbracket^{\mathbf{A}}(b, x, y) &= \begin{cases} x & \text{if } b = \mathbf{true} \\ y & \text{otherwise} \end{cases} \end{aligned}$$

and all other symbols from  $\Sigma_{\mathbf{Int}}$  are interpreted as usual (where  $\text{abs}(x)$  denotes the absolute value of  $x$ ).<sup>5,6</sup>

In [26], we used the semantics  $\llbracket \text{exp} \rrbracket^{\mathbf{A}}(x, y) = x^{|y|}$  instead. Throughout the paper, we will indicate all non-trivial changes that were required to adapt our approach to the new semantics

If  $\mathbf{A}$  is a  $\Sigma$ -structure and  $\Sigma'$  is a subsignature of  $\Sigma$ , then the *reduct* of  $\mathbf{A}$  to  $\Sigma'$  is the unique  $\Sigma'$ -structure that interprets its function symbols like  $\mathbf{A}$ . So the theory *non-linear integer arithmetic* (NIA) consists of the reducts of all elements of EIA to  $\Sigma_{\text{NIA}} = \Sigma_{\mathbf{Int}} \setminus \{\text{exp}\}$ .

Given a  $\Sigma$ -structure  $\mathbf{A}$  and a  $\Sigma$ -term  $t$ , the *meaning*  $\llbracket t \rrbracket^{\mathbf{A}}$  of  $t$  results from interpreting all function symbols according to  $\mathbf{A}$ . For function symbols  $f$  whose interpretation is fixed by a  $\Sigma$ -theory  $\mathcal{T}$ , we denote  $f$ 's interpretation by  $\llbracket f \rrbracket^{\mathcal{T}}$ . Given a  $\Sigma$ -theory  $\mathcal{T}$ , a  $\Sigma$ -formula  $\varphi$  (i.e., a  $\Sigma$ -term of sort **Bool**) is *satisfiable in*  $\mathcal{T}$  if there is an  $\mathbf{A} \in \mathcal{T}$  such that  $\llbracket \varphi \rrbracket^{\mathbf{A}} = \mathbf{true}$ . Then  $\mathbf{A}$  is called a *model* of  $\varphi$ , written  $\mathbf{A} \models \varphi$ . If every  $\mathbf{A} \in \mathcal{T}$  is a model of  $\varphi$ , then  $\varphi$  is  $\mathcal{T}$ -*valid*, written  $\models_{\mathcal{T}} \varphi$ . We write  $\psi \equiv_{\mathcal{T}} \varphi$  for  $\models_{\mathcal{T}} \psi \iff \varphi$ .

We sometimes also consider *uninterpreted functions*. Then the signature may not only contain the function symbols of the theory under consideration and variables, but also additional non-constant function symbols. As in SMT-LIB, we prefix theory names with “UF” to indicate that the signature may contain uninterpreted functions. So for example, UFNIA denotes the theory of non-linear integer arithmetic with uninterpreted functions.

We write “term”, “structure”, “theory”, ... instead of “ $\Sigma$ -term”, “ $\Sigma$ -structure”, “ $\Sigma$ -theory”, ... if  $\Sigma$  is irrelevant or clear from the context. Similarly, we just write “ $\equiv$ ” and “valid” instead of “ $\equiv_{\mathcal{T}}$ ” and “ $\mathcal{T}$ -valid” if  $\mathcal{T}$  is clear from the context. Moreover, we use unary minus and  $t^c$  (where  $t$  is a term of sort **Int** and  $c \in \mathbb{N}$ ) as syntactic sugar, and we write binary function symbols in infix notation.

In the sequel, we use  $x, y, z, \dots$  for variables,  $s, t, p, q, \dots$  for terms of sort **Int**,  $\varphi, \psi, \dots$  for formulas, and  $a, b, c, d, \dots$  for integers.

<sup>5</sup> Note that all functions are total in SMT-LIB logic. So `div` and `mod` are interpreted as integer division and modulo, respectively, if their second argument is non-zero. If the second argument is zero, then the interpretation of `div` and `mod` is arbitrary, i.e.,  $\lambda x. x \text{ div } 0$  and  $\lambda x. x \text{ mod } 0$  are treated like uninterpreted functions.

<sup>6</sup> The [SMT-LIB definition](#) of `exp` is slightly more technical, but equivalent. Moreover, the function symbol `ite` (“if-then-else”) is polymorphic in SMT-LIB. As we only need it for integers, a monomorphic variant suffices for our purposes. Similarly, the function symbol “`=`” is polymorphic in SMT-LIB. For simplicity, we use two monomorphic function symbols “`<=>`” (for **Bools**) and “`=`” (for **Ints**) instead.

**Algorithm 1: CEGAR for EIA**

```

Input: a  $\Sigma_{\text{Int}}^{\text{exp}}$ -formula  $\varphi$ 
// Preprocessing
1 do
2    $\varphi' \leftarrow \varphi$ 
3    $\varphi \leftarrow \text{FOLDCONSTANTS}(\varphi)$ 
4    $\varphi \leftarrow \text{REWRITE}(\varphi)$ 
5 while  $\varphi \neq \varphi'$ 
  // Refinement Loop
6    $Kinds \leftarrow \{\text{Negativity}, \text{Monotonicity}, \text{Bounding}, \text{Prime}, \text{Induction}, \text{Interpolation}\}$ 
7 while there is an UFNIA-model  $\mathbf{A}$  of  $\varphi$ 
8   if  $\mathbf{A}$  is a spurious counterexample then
9      $\mathcal{L} \leftarrow \emptyset$ 
10    for  $k \in Kinds$ 
11       $\mathcal{L} \leftarrow \mathcal{L} \cup \text{COMPUTELEMNAS}(\varphi, k)$ 
12     $\varphi \leftarrow \varphi \wedge \bigwedge \{\psi \in \mathcal{L} \mid \mathbf{A} \not\models \psi\}$ 
13  else return sat
14 return unsat

```

**3 Solving EIA Problems via CEGAR**

We now explain our technique for solving EIA problems, see Alg. 1. Our goal is to (dis)prove the satisfiability of  $\varphi$  in EIA. The loop in Line 7 is a CEGAR loop which lifts an SMT solver for UFNIA (which is called in Line 7) to EIA. So the *abstraction* consists of using UFNIA- instead of EIA-models. Hence, `exp` is considered to be an uninterpreted function in Line 7, i.e., the SMT solver also searches for an interpretation of `exp`.<sup>7</sup> If the model found by the SMT solver is a *spurious counterexample*, then the formula under consideration is refined by adding suitable lemmas in Lines 10 – 12 and the loop is iterated again.

**Definition 3.1 (Spurious Counterexample)** We call an UFNIA-model  $\mathbf{A}$  of  $\varphi$  a *spurious counterexample* if  $\mathbf{B} \not\models \varphi$ , where  $\mathbf{B}$  results from  $\mathbf{A}$  by replacing the interpretation of `exp` with  $\llbracket \text{exp} \rrbracket^{\text{EIA}}$ .

In [26], we used a stricter definition of spurious counterexamples which required that  $\mathbf{A}$  interprets every `exp`-term in  $\varphi$  according to the semantics of EIA. However, this is unnecessarily restrictive. Consider, e.g., the formula  $\text{exp}(2, x) \geq 4$  and the UFNIA-model  $\mathbf{A}$  with  $\llbracket x \rrbracket^{\mathbf{A}} = 2$  and  $\llbracket \text{exp}(2, x) \rrbracket^{\mathbf{A}} = 5$ . With the definition of spurious counterexamples from [26], we would have to perform a refinement step, as  $\llbracket \text{exp}(2, x) \rrbracket^{\mathbf{A}} = \llbracket \text{exp}(2, 2) \rrbracket^{\mathbf{A}} = 5 \neq \llbracket \text{exp}(2, 2) \rrbracket^{\text{EIA}}$ . With our new definition, we can immediately conclude satisfiability, as the interpretation  $\mathbf{B}$  that results from  $\mathbf{A}$  by interpreting `exp` correctly is an EIA-model.

In the sequel, we first discuss our preprocessings (first loop in Alg. 1) in Sect. 3.1. Then we explain our refinement (Lines 9 – 12) in Sect. 3.2. Here, we first introduce the different kinds of lemmas that are used by our implementation in Sect. 3.2.1 – 3.2.6. If implemented naively, the number of lemmas can get quite large, so we explain how to generate lemmas *lazily* in Sect. 3.3. Finally, we conclude this section by stating important properties of Alg. 1.

<sup>7</sup> For simplicity, in Alg. 1 we assume that the underlying UFNIA-solver either returns a model, proves that there is no model, or diverges (resulting in non-termination of Alg. 1). In practice, it may also return **unknown**, and then our implementation returns **unknown**, too.

*Example 3.2 (Leading Example)* To illustrate our approach, we show how to prove

$$\forall x, y. |x| \geq 3 \wedge |y| \geq 3 \implies \exp(\exp(x, y), y) \neq \exp(x, \exp(y, y)). \quad (3.1)$$

Here, we also consider negative values for  $x$  and  $y$  (i.e., we use the premise  $|x| \geq 3 \wedge |y| \geq 3$  instead of  $x \geq 3 \wedge y \geq 3$ ) to illustrate how our technique works when  $\exp$  is used with negative arguments. To prove (3.1), we prove the unsatisfiability of its negation:

$$\text{abs}(x) \geq 3 \wedge \text{abs}(y) \geq 3 \wedge \exp(\exp(x, y), y) = \exp(x, \exp(y, y))$$

### 3.1 Preprocessings

In the first loop of Alg. 1, we preprocess  $\varphi$  by alternating *constant folding* (Line 3) and *rewriting* (Line 4) until a fixpoint is reached. Constant folding evaluates subexpressions without variables, where subexpressions  $\exp(c, d)$  are evaluated according to the semantics of EIA. Rewriting simplifies  $\exp$ -terms via the following (terminating) rewrite rules:

$$\exp(x, c) \rightarrow x^c \quad \text{if } c \in \mathbb{N}$$

$$\exp(\exp(x, y), z) \rightarrow \text{ite}(y < 0 \wedge z < 0, \exp(x, -(y \cdot z)), \exp(x, y \cdot z)) \quad (3.2)$$

$$\exp(x, y) \cdot \exp(x, z) \rightarrow \text{ite}(\text{abs}(x) \neq 1 \wedge (y < 0 \vee z < 0), 0, \exp(x, y + z)) \quad (3.3)$$

$$\exp(x, y) \cdot \exp(z, y) \rightarrow \exp(x \cdot z, y)$$

In particular, the 1<sup>st</sup> rule allows us to rewrite<sup>8</sup>  $\exp(s, 0)$  to  $s^0 = 1$  and  $\exp(s, 1)$  to  $s^1 = s$ .<sup>9</sup> Note that the rule

$$\exp(\exp(x, y), z) \rightarrow \exp(x, y \cdot z)$$

from [26] is unsound with the new semantics of  $\exp$ , as we have, e.g.,

$$\llbracket \exp \rrbracket^{\text{EIA}} (\llbracket \exp \rrbracket^{\text{EIA}} (2, -2), -2) = 0 \neq 16 = \llbracket \exp \rrbracket^{\text{EIA}} (2, 4).$$

Thus, we use the rewrite rule (3.2) instead, which fixes the problematic case where both exponents on the left-hand side are negative by explicitly negating the exponent on the right-hand side. Similarly, the rule

$$\exp(x, y) \cdot \exp(x, z) \rightarrow \exp(x, y + z)$$

would be unsound, as we have, e.g.,

$$\llbracket \exp \rrbracket^{\text{EIA}} (2, -2) \cdot \llbracket \exp \rrbracket^{\text{EIA}} (2, 2) = 0 \neq 1 = \llbracket \exp \rrbracket^{\text{EIA}} (2, 0).$$

Thus, we use the rule (3.3) instead.

*Example 3.3 (Preprocessing)* For our leading example, applying the rewrite rule (3.2) at the underlined position yields:

$$\begin{aligned} & \text{abs}(x) \geq 3 \wedge \text{abs}(y) \geq 3 \wedge \underline{\exp(\exp(x, y), y)} = \exp(x, \exp(y, y)) \\ \rightarrow & \text{abs}(x) \geq 3 \wedge \text{abs}(y) \geq 3 \wedge \text{ite}(y < 0, \exp(x, -y^2), \exp(x, y^2)) = \exp(x, \exp(y, y)) \quad (\dagger) \end{aligned}$$

**Lemma 3.4** We have

$$\varphi \equiv_{\text{EIA}} \text{FOLDCONSTANTS}(\varphi) \quad \text{and} \quad \varphi \equiv_{\text{EIA}} \text{REWRITE}(\varphi).$$

<sup>8</sup> Note that we have  $\llbracket \exp(0, 0) \rrbracket^{\text{EIA}} = 0^0 = 1$ .

<sup>9</sup> Clearly, the first rewrite rule may introduce large-degree polynomials. In our implementation, we nevertheless apply it unconditionally and exhaustively.

### 3.2 Refinement

Our refinement (Lines 9 – 12 of Alg. 1) is based on the six kinds of lemmas named in Line 6: *negativity lemmas*, *monotonicity lemmas*, *bounding lemmas*, *prime lemmas*, *induction lemmas*, and *interpolation lemmas*. In the sequel, we explain how we compute a set  $\mathcal{L}$  of such lemmas. Then our refinement conjoins

$$\{\psi \in \mathcal{L} \mid \mathbf{A} \not\models \psi\}$$

to  $\varphi$  in Line 12. As our lemmas allow us to eliminate *any* spurious counterexample, this set is never empty, see Thm. 3.26. To compute  $\mathcal{L}$ , we consider all terms that are *relevant* for the formula  $\varphi$ .

**Definition 3.5 (Relevant Terms)** A term  $\exp(s, t)$  is *relevant* if  $\varphi$  has a subterm of the form  $\exp(\pm s, t)$ .

*Example 3.6 (Relevant Terms)* For our leading example ( $\dagger$ ), the relevant terms are all terms of the form

$$\exp(\pm x, \pm y^2), \quad \exp(\pm y, y), \quad \text{or} \quad \exp(\pm x, \exp(y, y)).$$

While the formula  $\varphi$  is changed in Line 12 of Alg. 1, we only conjoin new lemmas to  $\varphi$ , and thus relevant terms can never become irrelevant. Moreover, by construction our lemmas only contain  $\exp$ -terms that were already relevant before. Thus, the set of relevant terms is not changed by our CEGAR loop.

As mentioned in Sect. 1, our approach may also compute lemmas with non-linear polynomial arithmetic. However, our lemmas are linear if  $s$  is an integer constant and  $t$  is linear for all subterms  $\exp(s, t)$  of  $\varphi$ .

#### 3.2.1 Negativity Lemmas

*Negativity lemmas* handle terms of the form  $\exp(s, t)$  where  $s$  or  $t$  is negative. For each relevant term  $\exp(s, t)$ , the set  $\mathcal{L}$  contains the following negativity lemmas:

$$\begin{aligned} \text{divisible}_2(t) &\implies \exp(s, t) = \exp(-s, t) && (\text{NEG}_1) \\ \neg \text{divisible}_2(t) &\implies \exp(s, t) = -\exp(-s, t) && (\text{NEG}_2) \\ \text{abs}(s) \neq 1 \wedge t < 0 &\implies \exp(s, t) = 0 && (\text{NEG}_3) \end{aligned}$$

Note that  $\text{NEG}_1$  and  $\text{NEG}_2$  are just implications, not equivalences (even if one only considers non-negative exponents), as, for example,  $c^d = (-c)^d$  does not imply  $\text{divisible}_2(d)$  if  $c = 0$ . Moreover, note that the first two lemmas reduce the case  $s = -1 \wedge t < 0$  (which is not covered by  $\text{NEG}_3$ ) to the case  $\exp(1, t)$ , which in turn is covered by *bounding lemmas*, see Sect. 3.2.3.

In [26], we used *symmetry lemmas* to handle negative arguments of  $\exp$ . While  $\text{NEG}_1$  and  $\text{NEG}_2$  still encode the symmetric nature of  $\exp$  w.r.t. its first argument,  $\exp$  is no longer symmetric w.r.t. its second argument with the new semantics. Hence, we introduced  $\text{NEG}_3$  to handle negative exponents.

*Example 3.7 (Negativity Lemmas)* For our leading example ( $\dagger$ ), the following negativity lemmas would be considered, among others:

$$\begin{aligned}
\text{NEG}_1[s/x, t/y^2] : \quad & \text{divisible}_2(y^2) \implies \exp(x, y^2) = \exp(-x, y^2) \\
\text{NEG}_2[s/-x, t/y^2] : \quad & \neg \text{divisible}_2(y^2) \implies \exp(-x, y^2) = -\exp(x, y^2) \\
\text{NEG}_3[s/y, t/y] : \quad & \text{abs}(y) \neq 1 \wedge y < 0 \implies \exp(y, y) = 0 \quad (3.4)
\end{aligned}$$

$$\text{NEG}_3[s/x, t/-y^2] : \quad \text{abs}(x) \neq 1 \wedge -y^2 < 0 \implies \exp(x, -y^2) = 0 \quad (3.5)$$

Note that, e.g., the second lemma above results from the term  $\exp(-x, y^2)$ , which is relevant (see Def. 3.5) even though it does not occur in  $\varphi$ .

To prove the soundness of our refinement, we have to show that our lemmas are EIA-valid.

**Lemma 3.8** Let  $s, t$  be terms of sort **Int**. Then  $\text{NEG}_1 - \text{NEG}_3$  are EIA-valid.

### 3.2.2 Monotonicity Lemmas

*Monotonicity lemmas* are of the form

$$s_2 \geq s_1 \geq 2 \wedge t_2 \geq t_1 > 0 \wedge (s_2 > s_1 \vee t_2 > t_1) \implies \exp(s_2, t_2) > \exp(s_1, t_1), \text{ (MON)}$$

i.e., they prohibit violations of the monotonicity of  $\exp$ .

*Example 3.9 (Monotonicity Lemmas)* For our leading example ( $\dagger$ ), we obtain, e.g., the lemma  $\text{MON}[s_1/x, t_1/y^2, s_2/x, t_2/\exp(y, y)]$ :<sup>10</sup>

$$x \geq 2 \wedge \exp(y, y) > y^2 > 0 \implies \exp(x, \exp(y, y)) > \exp(x, y^2) \quad (3.6)$$

Thus, for each pair of two different relevant terms  $\exp(s_1, t_1), \exp(s_2, t_2)$  where  $\llbracket s_2 \rrbracket \geq \llbracket s_1 \rrbracket \geq 2$  and  $\llbracket t_2 \rrbracket \geq \llbracket t_1 \rrbracket > 0$ , the set  $\mathcal{L}$  contains  $\text{MON}$ . Here and in the sequel, unless mentioned otherwise,  $\llbracket \dots \rrbracket$  means  $\llbracket \dots \rrbracket^{\mathbf{A}}$ , where  $\mathbf{A}$  is the model from Line 7 of Alg. 1.

**Lemma 3.10** Let  $s_1, s_2, t_1, t_2$  be terms of sort **Int**. Then  $\text{MON}$  is EIA-valid.

### 3.2.3 Bounding Lemmas

*Bounding lemmas* provide bounds on relevant terms  $\exp(s, t)$ . For each relevant term  $\exp(s, t)$ , the following lemmas are contained in  $\mathcal{L}$ :

$$\begin{aligned}
t = 0 & \implies \exp(s, t) = 1 & (\text{BND}_1) \\
t = 1 & \implies \exp(s, t) = s & (\text{BND}_2) \\
s = 0 \wedge t \neq 0 & \implies \exp(s, t) = 0 & (\text{BND}_3) \\
s = 1 & \implies \exp(s, t) = 1 & (\text{BND}_4) \\
s + t \geq 5 \wedge s \geq 2 \wedge t \geq 2 & \implies \exp(s, t) > s \cdot t + 1 & (\text{BND}_5)
\end{aligned}$$

The cases  $t \in \{0, 1\}$  are also addressed by our first rewrite rule (see Sect. 3.1). However, this rewrite rule only applies if  $t$  is an integer constant. In contrast, the first two lemmas above apply if  $t$  evaluates to 0 or 1 in the current model. The bounding lemmas are defined in such a way that they provide lower bounds for

<sup>10</sup> Here and in the sequel, we simplify lemmas equivalently in examples for the sake of readability.

$\exp(s, t)$  for almost all non-negative values of  $s$  and  $t$ : **BND<sub>1</sub>–BND<sub>4</sub>** provide tight bounds for all cases where at least one argument of  $\exp$  is 0 or 1, and **BND<sub>5</sub>** provides lower bounds for all other cases except for  $\exp(2, 2)$ . To cover *all* cases, we could replace **BND<sub>5</sub>** by

$$s \geq 2 \wedge t \geq 0 \implies \exp(s, t) \geq s \cdot t.$$

However, in contrast to the alternative lemma above, **BND<sub>5</sub>** expresses that  $\exp(s, t)$  is *strictly* greater than  $s \cdot t$ . The reason why the bounding lemmas focus on lower bounds is that polynomials can only bound  $\exp(s, t)$  from above for finitely many values of  $s$  and  $t$ . Together with negativity lemmas, bounding lemmas also give rise to bounds for the cases where  $s$  or  $t$  are negative. The missing (lower and upper) bounds are provided by *interpolation lemmas* (see Sect. 3.2.6).

Compared to [26], **BND<sub>3</sub>** is just an implication, not an equivalence, as  $\exp$  can now also be zero for non-zero bases if the second argument is negative.

*Example 3.11 (Bounding Lemmas)* For our leading example ( $\dagger$ ), the following bounding lemmas would be considered, among others:

$$\begin{aligned} \text{BND}_1[s/x, t/\exp(y, y)] : & \quad \exp(y, y) = 0 \implies \exp(x, \exp(y, y)) = 1 & (3.7) \\ \text{BND}_2[s/x, t/\exp(y, y)] : & \quad \exp(y, y) = 1 \implies \exp(x, \exp(y, y)) = x \\ \text{BND}_3[s/x, t/\exp(y, y)] : & \quad x = 0 \wedge \exp(y, y) \neq 0 \implies \exp(x, \exp(y, y)) = 0 \\ \text{BND}_4[s/x, t/\exp(y, y)] : & \quad x = 1 \implies \exp(x, \exp(y, y)) = 1 \\ \text{BND}_5[s/y, t/y] : & \quad y \geq 3 \implies \exp(y, y) > y^2 + 1 & (3.8) \end{aligned}$$

**Lemma 3.12** Let  $s, t$  be terms of sort **Int**. Then **BND<sub>1</sub> – BND<sub>5</sub>** are EIA-valid.

### 3.2.4 Prime Lemmas

*Prime lemmas* are of the form

$$t > 0 \implies (\text{divisible}_d(\exp(s, t)) \iff \text{divisible}_d(s)) \quad \text{where } d \text{ is prime.} \quad (\text{PRIME})$$

So prime lemmas express that  $\exp(s, t)$  and  $s$  have the same prime factors if  $t > 0$ .

*Example 3.13 (Prime Lemmas)* Consider the formula

$$y > 0 \wedge \exp(2, x) = \exp(3, y), \quad (3.9)$$

which we used in [26] (where prime lemmas were not yet integrated) to illustrate the incompleteness of our approach.<sup>11</sup> The unsatisfiability of this formula can easily be proven with the following two prime lemmas, which rule out structures **A** where  $\mathbf{A} \models x > 0$ :

$$\begin{aligned} \text{PRIME}[s/2, t/x] : & \quad x > 0 \implies (\text{divisible}_2(\exp(2, x)) \iff \text{divisible}_2(2)) \\ & \quad \equiv x > 0 \implies \text{divisible}_2(\exp(2, x)) \\ \text{PRIME}[s/3, t/y] : & \quad y > 0 \implies (\text{divisible}_2(\exp(3, y)) \iff \text{divisible}_2(3)) \\ & \quad \equiv y > 0 \implies \neg \text{divisible}_2(\exp(3, y)) \end{aligned}$$

<sup>11</sup> More precisely, the corresponding example in [26] was  $y \neq 0 \wedge \exp(2, x) = \exp(3, y)$ , which is, however, satisfiable with the new semantics of  $\exp$  (as we have, e.g.,  $\exp(2, -1) = 0 = \exp(3, -1)$ ).

The reason is that  $x > 0$  implies  $\text{divisible}_2(\text{exp}(2, x))$  by the first prime lemma, and  $y > 0$  and the second prime lemma imply  $\neg \text{divisible}_2(\text{exp}(3, y))$ . Thus,  $\text{exp}(2, x) \neq \text{exp}(3, y)$  follows. For the case  $x \leq 0$ , the following additional lemmas are needed:

$$\begin{array}{ll} \text{NEG}_3[s/2, t/x] : & x < 0 \implies \text{exp}(2, x) = 0 \\ \text{BND}_1[s/2, t/x] : & x = 0 \implies \text{exp}(2, x) = 1 \\ \text{BND}_2[s/3, t/y] : & y = 1 \implies \text{exp}(3, y) = 3 \\ \text{BND}_5[s/3, t/y] : & y \geq 2 \implies \text{exp}(3, y) > 3 \cdot y + 1 \end{array}$$

Then we have  $\text{exp}(2, x) \leq 1$  by the first two lemmas and  $\text{exp}(3, y) \geq 3$  for all  $y > 0$  by the third and fourth lemma. As (3.9) implies  $y > 0$ , this suffices to prove unsatisfiability.

Thus, for each relevant term  $\text{exp}(s, t)$  where  $\llbracket s \rrbracket \geq 2$ ,  $\llbracket \text{exp}(s, t) \rrbracket \geq 2$ , and where the sets of prime factors of  $\llbracket s \rrbracket$  and  $\llbracket \text{exp}(s, t) \rrbracket$  differ,  $\mathcal{L}$  contains **PRIME**, where  $d$  is the smallest prime which divides  $\llbracket s \rrbracket$  or  $\llbracket \text{exp}(s, t) \rrbracket$ , but not both of them. To find  $d$ , we use *wheel factorization* [41].

Wheel factorization is based on the observation that, when searching for the smallest prime factor of a given number, it suffices to first test for divisibility by 2, and then iteratively test for divisibility by odd numbers  $\geq 3$  in increasing order. To further improve efficiency, one can first test for divisibility by 2 and 3, and then iteratively test for divisibility by integers  $\geq 4$  that are coprime with 2 and 3. In general, one can initially test for divisibility by the first  $c$  prime numbers  $p_1, \dots, p_c$ , and then iteratively test for divisibility by integers  $> p_c$  that are coprime with  $p_1, \dots, p_c$ .

Thus, to find  $d$ , we first compute the greatest common divisor  $g$  of  $\llbracket \text{exp}(s, t) \rrbracket$  and  $\llbracket s \rrbracket$ . Then  $a = \frac{\llbracket \text{exp}(s, t) \rrbracket}{g}$  and  $b = \frac{\llbracket s \rrbracket}{g}$  are coprime, so it suffices to find the smallest number which is a prime factor of either  $a$  or  $b$  via wheel factorization. In our implementation of wheel factorization, we initially test for divisibility by 2, 3, and 5.

**Lemma 3.14** Let  $s, t$  be terms of sort **Int** and let  $d \in \mathbb{N}$  be prime. Then **PRIME** is EIA-valid.

### 3.2.5 Induction Lemmas

*Induction lemmas* are of the form

$$s_1 = s_2 \wedge t_2 - d = t_1 \geq 0 \implies \text{exp}(s_2, t_2) = \text{exp}(s_1, t_1) \cdot s_1^d \quad \text{where } d \in \mathbb{N}_{>0}. \quad (\text{IND})$$

So intuitively, **IND** results from  $\text{exp}(s_2, t_2)$  by  $d$  unrollings of the recursive definition

$$\text{exp}(s, x) = \begin{cases} \text{exp}(s, x-1) \cdot s & \text{if } x > 0 \\ 1 & \text{if } x = 0 \end{cases}$$

as follows:

$$\begin{aligned}
\exp(s_2, t_2) &= \exp(s_2, t_2 - 1) \cdot s_2 \\
&= \exp(s_2, t_2 - 2) \cdot s_2^2 \\
&= \dots \\
&= \exp(s_2, t_2 - d) \cdot s_2^d \\
&= \exp(s_1, t_1) \cdot s_1^d
\end{aligned}$$

*Example 3.15 (Induction Lemmas)* We demonstrate the usefulness of induction lemmas by an example where SMT solving is used to verify a solution for a recurrence relation. Consider the following recurrence relation (where  $n$  ranges over  $\mathbb{N}_{>0}$ ):

$$f(n) = 2 \cdot f(n-1) + 2^n \quad (3.10)$$

A possible solution (i.e., a function  $f$  that satisfies (3.10)) is

$$f(n) = (f(0) + n) \cdot 2^n. \quad (3.11)$$

To verify that this function is indeed a solution, it suffices to replace  $f$  with the definition from (3.11) and show that the resulting equation holds for all  $n \in \mathbb{N}_{>0}$  and all  $f(0) \in \mathbb{Z}$ , i.e., to prove

$$\forall n \in \mathbb{N}_{>0}, f_0 \in \mathbb{Z}. (f_0 + n) \cdot 2^n = 2 \cdot (f_0 + n - 1) \cdot 2^{n-1} + 2^n.$$

To do so, it suffices to prove the unsatisfiability of

$$n > 0 \wedge (f_0 + n) \cdot \exp(2, n) \neq 2 \cdot (f_0 + n - 1) \cdot \exp(2, n - 1) + \exp(2, n). \quad (3.12)$$

To this end, our implementation deduces the following induction lemma:

$$\text{IND}[s_1/2, t_1/n - 1, s_2/2, t_2/n] : \quad n > 0 \implies \exp(2, n) = 2 \cdot \exp(2, n - 1)$$

Then unsatisfiability can easily be proven without reasoning about  $\exp$ : Assume  $n > 0$ . Then by the induction lemma, we may substitute  $\exp(2, n)$  with  $2 \cdot \exp(2, n - 1)$ , resulting in

$$\begin{aligned}
&(f_0 + n) \cdot 2 \cdot \exp(2, n - 1) \neq 2 \cdot (f_0 + n - 1) \cdot \exp(2, n - 1) + 2 \cdot \exp(2, n - 1) \\
&\equiv (f_0 + n) \cdot 2 \cdot \exp(2, n - 1) \neq 2 \cdot (f_0 + n) \cdot \exp(2, n - 1)
\end{aligned}$$

which is trivially unsatisfiable. Hence, (3.11) is a valid solution for (3.10).

Thus, for each pair of relevant terms  $\exp(s_1, t_1), \exp(s_2, t_2)$  where  $\llbracket s_1 \rrbracket = \llbracket s_2 \rrbracket$  and  $\llbracket t_2 \rrbracket - d = \llbracket t_1 \rrbracket \geq 0$  for some  $d \in \mathbb{N}_{>0}$ ,  $\mathcal{L}$  contains  $\text{IND}$ .<sup>12</sup>

**Lemma 3.16** Let  $s_1, s_2, t_1, t_2$  be terms of sort **Int** and let  $d \in \mathbb{N}_{>0}$ . Then  $\text{IND}$  is EIA-valid.

<sup>12</sup> Note that the factor  $s_1^d$  grows exponentially w.r.t.  $d$ . Nevertheless, our implementation does not impose a limit on  $d$  or  $s_1^d$ .

### 3.2.6 Interpolation Lemmas

To provide bounds in cases where no bounding lemmas are violated, we use *interpolation lemmas* that are constructed via *bilinear interpolation*. Here, we assume that the arguments of  $\exp$  are positive, as negative arguments are handled by negativity lemmas, and bounding lemmas yield tight bounds if at least one argument of  $\exp$  is 0. The correctness of interpolation lemmas relies on the following observation. Here, as usual,  $[w_1, w_2]$  and  $(w_1, w_2)$  denote closed and open real intervals.

**Lemma 3.17** Let  $c \geq 0$ ,  $w_1, w_2 \in \mathbb{R}_{>c}$ ,<sup>13</sup>  $w_1 < w_2$ , and let  $f : \mathbb{R}_{>c} \rightarrow \mathbb{R}_{>0}$  be convex. Then

$$\begin{aligned} \forall x \in [w_1, w_2]. \quad f(x) &\leq f(w_1) + \frac{f(w_2) - f(w_1)}{w_2 - w_1} \cdot (x - w_1) \quad \text{and} \\ \forall x \in \mathbb{R}_{>c} \setminus (w_1, w_2). \quad f(x) &\geq f(w_1) + \frac{f(w_2) - f(w_1)}{w_2 - w_1} \cdot (x - w_1). \end{aligned}$$

Note that the right-hand side of the inequations above is the linear interpolant of  $f$  between  $w_1$  and  $w_2$ . Intuitively, it corresponds to the secant of  $f$  between the points  $(w_1, f(w_1))$  and  $(w_2, f(w_2))$ , and thus the lemma follows from the convexity of  $f$ .

Let  $\exp(s, t)$  be relevant,  $\llbracket s \rrbracket = c > 0$ ,  $\llbracket t \rrbracket = d > 0$ , and  $\llbracket \exp \rrbracket(c, d) \neq c^d$ , i.e., we want to prohibit the current interpretation of  $\exp(s, t)$ .

*Interpolation Lemmas for Upper Bounds* First assume  $\llbracket \exp \rrbracket(c, d) > c^d$ , i.e., the value of  $\exp(c, d)$  in the current model  $\mathbf{A}$  is too large. So to rule out this spurious counterexample, we need a lemma that provides a suitable upper bound for  $\exp(c, d)$ . Let  $c', d' \in \mathbb{N}_{>0}$  and:

$$\begin{aligned} c^- &:= \min(c, c') & c^+ &:= \max(c, c') & d^- &:= \min(d, d') & d^+ &:= \max(d, d') \\ [c^\pm] &:= [c^- .. c^+] & [d^\pm] &:= [d^- .. d^+] \end{aligned}$$

Here,  $[a .. b]$  denotes a closed integer interval. Then we first use  $d^-, d^+$  for linear interpolation w.r.t. the  $2^{nd}$  argument of  $\lambda x, y. x^y$ . To this end, let

$$\text{ip}_2^{[d^\pm]}(x, y) := x^{d^-} + \frac{x^{d^+} - x^{d^-}}{d^+ - d^-} \cdot (y - d^-),$$

where we define  $\frac{a}{b} := \frac{a}{b}$  if  $b \neq 0$  and  $\frac{a}{0} := 0$ .<sup>14</sup> So if  $d^- < d^+$ , then  $\text{ip}_2^{[d^\pm]}(x, y)$  corresponds to the linear interpolant of  $x^y$  w.r.t.  $y$  between  $d^-$  and  $d^+$ . Then  $\text{ip}_2^{[d^\pm]}(x, y)$  is a suitable upper bound, as

$$\forall x \in \mathbb{N}_{>0}, y \in [d^\pm]. \quad x^y \leq \text{ip}_2^{[d^\pm]}(x, y) \quad (3.13)$$

<sup>13</sup> The lemma also holds for  $\mathbb{R}_{\geq c}$  (and the proof is analogous).

<sup>14</sup> Note that our definition of  $\frac{a}{0}$  is in contrast to the handling of division by zero in SMT-LIB. There, division by zero is left uninterpreted in order to model partial functions (since SMT-LIB logic only allows total functions). In contrast, our handling of  $\frac{a}{0}$  is due to the fact that our interpolation lemmas would not have the intended semantics for the case  $d^- = d^+$ , otherwise.

follows from Lemma 3.17: If  $d^- = d^+$ , then (3.13) is trivial. Otherwise, let  $x \in \mathbb{N}_{>0}$  be arbitrary but fixed. Then (3.13) follows by applying Lemma 3.17 to  $f(y) := x^y$ , which is clearly convex on  $\mathbb{R}_{>0}$ .

Hence, we could derive the following EIA-valid lemma:<sup>15</sup>

$$s > 0 \wedge t \in [d^\pm] \implies \exp(s, t) \leq \text{ip}_2^{[d^\pm]}(s, t) \quad (\text{IP}_2)$$

Strictly speaking, this lemma is not a  $\Sigma_{\text{Int}}^{\text{exp}}$ -term if  $d^+ > d^-$ , as the right-hand side makes use of (rational) division in this case. However, an equivalent  $\Sigma_{\text{Int}}^{\text{exp}}$ -term can clearly be obtained by multiplying with the divisor. Here and in the sequel, we use division in such cases to ease readability.

*Example 3.18 (Linear Interpolation w.r.t. y)* Let  $\llbracket \exp(s, t) \rrbracket = \llbracket \exp \rrbracket(3, 9) > 3^9$ , i.e., we have  $c = 3$  and  $d = 9$ . Moreover, assume  $c' = d' = 1$ , i.e., we get  $c^- = 1$ ,  $c^+ = 3$ ,  $d^- = 1$ , and  $d^+ = 9$ . Then

$$\text{ip}_2^{[d^\pm]}(x, y) = \text{ip}_2^{[1..9]}(x, y) = x^1 + \frac{x^9 - x^1}{9 - 1} \cdot (y - 1) = x + \frac{x^9 - x}{8} \cdot (y - 1).$$

Hence,  $\text{IP}_2$  corresponds to

$$s > 0 \wedge t \in [1, 9] \implies \exp(s, t) \leq s + \frac{s^9 - s}{8} \cdot (t - 1).$$

This lemma would be violated by our spurious counterexample, as we have

$$\left\lceil s + \frac{s^9 - s}{8} \cdot (t - 1) \right\rceil = 3 + \frac{3^9 - 3}{8} \cdot 8 = 3^9 < \llbracket \exp \rrbracket(3, 9) = \llbracket \exp(s, t) \rrbracket.$$

However, the degree of  $\text{ip}_2^{[d^\pm]}(s, t)$  depends on  $d^+$ , which in turn depends on the model that was found by the underlying SMT solver. Thus, the degree of  $\text{ip}_2^{[d^\pm]}(s, t)$  can get very large, which is challenging for the underlying solver.

So we next use  $c^-, c^+$  for linear interpolation w.r.t. the  $1^{st}$  argument of  $\lambda x, y. x^y$ , resulting in

$$\text{ip}_1^{[c^\pm]}(x, y) := (c^-)^y + \frac{(c^+)^y - (c^-)^y}{c^+ - c^-} \cdot (x - c^-).$$

Then due to Lemma 3.17,  $\text{ip}_1^{[c^\pm]}(x, y)$  is also an upper bound on the exponentiation function, i.e., we have

$$\forall y \in \mathbb{N}_{>0}, x \in [c^\pm]. x^y \leq \text{ip}_1^{[c^\pm]}(x, y). \quad (3.14)$$

To see this, note that (3.14) is trivial if  $c^- = c^+$ . Otherwise, let  $y \in \mathbb{N}_{>0}$  be arbitrary but fixed. Then (3.14) follows by applying Lemma 3.17 to  $f(x) := x^y$ , which is clearly convex on  $\mathbb{R}_{>0}$ .

Note that we have  $\frac{y - d^-}{d^+ - d^-} \in [0, 1]$  for all  $y \in [d^\pm]$ , and thus

$$\text{ip}_2^{[d^\pm]}(x, y) = x^{d^-} \cdot \left( 1 - \frac{y - d^-}{d^+ - d^-} \right) + x^{d^+} \cdot \frac{y - d^-}{d^+ - d^-}$$

<sup>15</sup> We only give  $\text{IP}_2$  for illustration purposes. Our implementation does not synthesize such lemmas, but it uses  $\text{IP}_{\text{up}}$  instead, which is better suited for automation.

is monotonically increasing in both  $x^{d^-}$  and  $x^{d^+}$ . Hence, in the definition of  $\text{ip}_2^{[d^\pm]}$ , we can approximate  $x^{d^-}$  and  $x^{d^+}$  with their upper bounds  $\text{ip}_1^{[c^\pm]}(x, d^-)$  and  $\text{ip}_1^{[c^\pm]}(x, d^+)$  that can be derived from (3.14). Then (3.13) yields

$$\forall x \in [c^\pm], y \in [d^\pm]. \quad x^y \leq \text{ip}^{[c^\pm][d^\pm]}(x, y) \quad (3.15)$$

where

$$\text{ip}^{[c^\pm][d^\pm]}(x, y) := \text{ip}_1^{[c^\pm]}(x, d^-) + \frac{\text{ip}_1^{[c^\pm]}(x, d^+) - \text{ip}_1^{[c^\pm]}(x, d^-)}{d^+ - d^-} \cdot (y - d^-).$$

So the set  $\mathcal{L}$  contains the lemma

$$s \in [c^\pm] \wedge t \in [d^\pm] \implies \exp(s, t) \leq \text{ip}^{[c^\pm][d^\pm]}(s, t), \quad (\text{IP}_{\text{up}})$$

which is valid due to (3.15), and rules out any spurious counterexample with  $\llbracket \exp \rrbracket(c, d) > c^d$ , as  $\text{ip}^{[c^\pm][d^\pm]}(c, d) = c^d$ .

**Lemma 3.19** Let  $c^+ \geq c^- > 0$  and  $d^+ \geq d^- > 0$ . Then  $\text{IP}_{\text{up}}$  is EIA-valid.

*Example 3.20 (Bilinear Interpolation, Ex. 3.18 continued)* In our example, we have:

$$\begin{aligned} \text{ip}_1^{[c^\pm]}(x, y) &= \text{ip}_1^{[1..3]}(x, y) = 1^y + \frac{3^y - 1^y}{3 - 1} \cdot (x - 1) = 1 + \frac{3^y - 1}{2} \cdot (x - 1) \\ \text{ip}_1^{[c^\pm]}(s, d^-) &= \text{ip}_1^{[1..3]}(s, 1) = 1 + \frac{3 - 1}{2} \cdot (s - 1) = s \\ \text{ip}_1^{[c^\pm]}(s, d^+) &= \text{ip}_1^{[1..3]}(s, 9) = 1 + \frac{3^9 - 1}{2} \cdot (s - 1) = 1 + 9841 \cdot (s - 1) \end{aligned}$$

Hence, we obtain the lemma

$$s \in [1, 3] \wedge t \in [1, 9] \implies \exp(s, t) \leq s + \frac{1 + 9841 \cdot (s - 1) - s}{8} \cdot (t - 1).$$

This lemma is violated by our spurious counterexample, as we have

$$\left\llbracket s + \frac{1 + 9841 \cdot (s - 1) - s}{8} \cdot (t - 1) \right\rrbracket = 3^9 < \llbracket \exp \rrbracket(3, 9) = \llbracket \exp(s, t) \rrbracket.$$

$\text{IP}_{\text{up}}$  relates  $\exp(s, t)$  with the *bilinear* function  $\text{ip}^{[c^\pm][d^\pm]}(s, t)$ , i.e., this function is linear w.r.t. both  $s$  and  $t$ , but it multiplies  $s$  and  $t$ . Thus, if  $s$  is an integer constant and  $t$  is linear, then the resulting lemma is linear, too.

To compute interpolation lemmas, a second point  $(c', d')$  is needed. Thus, for each relevant  $\exp$ -term, our implementation maintains a store of all points  $(c, d)$  where interpolation has been applied previously,<sup>16</sup> and uses the one which is closest to the current one (in terms of the Euclidean distance). The same heuristic is used by Cimatti et al. [15] to compute *secant lemmas*. For the 1<sup>st</sup> interpolation step, we choose  $(c', d') = (c, d)$ . In this case,  $\text{IP}_{\text{up}}$  simplifies to  $s = c \wedge t = d \implies \exp(s, t) \leq c^d$ .

<sup>16</sup> Our implementation does not reset or prune this store at any point.

*Interpolation Lemmas for Lower Bounds* While bounding lemmas already yield lower bounds, the bounds provided by **BND<sub>5</sub>** are not exact, in general. Hence, if  $\llbracket \text{exp} \rrbracket(c, d) < c^d$ , then we also use bilinear interpolation to obtain a precise lower bound for  $\text{exp}(c, d)$ . Dually to (3.13) and (3.14), Lemma 3.17 implies:

$$\forall x, y \in \mathbb{N}_{>0}. x^y \geq \text{ip}_2^{[d..d+1]}(x, y) \quad (3.16)$$

$$\forall x, y \in \mathbb{N}_{>0}. x^y \geq \text{ip}_1^{[c..c+1]}(x, y) \quad (3.17)$$

To see this for (3.16), let  $x \in \mathbb{N}_{>0}$  be arbitrary but fixed. Then (3.16) follows by applying Lemma 3.17 to  $f(y) := x^y$ , which is clearly convex on  $\mathbb{R}_{>0}$ . While Lemma 3.17 implies the claim for  $y \in \mathbb{R}_{>0} \setminus (d, d+1)$ , note that  $\mathbb{N}_{>0} \subseteq \mathbb{R}_{>0} \setminus (d, d+1)$ , since  $(d, d+1)$  does not contain any integers. The proof of (3.17) works analogously by considering  $f(x) := x^y$  for fixed  $y \in \mathbb{N}_{>0}$ .

Additionally, we also obtain

$$\forall x, y \in \mathbb{N}_{>0}. x^{y+1} - x^y \geq \text{ip}_1^{[c..c+1]}(x, y+1) - \text{ip}_1^{[c..c+1]}(x, y) \quad (3.18)$$

from Lemma 3.17. The reason is that for  $f(x) := x^{y+1} - x^y$ , the right-hand side of (3.18) is equal to the linear interpolant of  $f$  between  $c$  and  $c+1$ . Moreover,  $f$  is convex, as  $f(x) = x^y \cdot (x-1)$  where for any fixed  $y \in \mathbb{N}_{>0}$ , both  $x^y$  and  $x-1$  are non-negative, monotonically increasing, and convex on  $\mathbb{R}_{\geq 1}$ . Thus, (3.18) follows by applying Lemma 3.17 to  $f(x) := x^{y+1} - x^y$ , since  $\mathbb{N}_{>0} \subseteq \mathbb{R}_{\geq 1} \setminus (c, c+1)$ .

If  $y \geq d$ , then  $\text{ip}_2^{[d..d+1]}(x, y) = x^d + (x^{d+1} - x^d) \cdot (y-d)$  is monotonically increasing in the first occurrence of  $x^d$ , and in  $x^{d+1} - x^d$ . Thus, by approximating  $x^d$  and  $x^{d+1} - x^d$  with their lower bounds from (3.17) and (3.18), (3.16) yields

$$\begin{aligned} \forall x \in \mathbb{N}_{>0}, y \geq d. \\ x^y &\geq \text{ip}_1^{[c..c+1]}(x, d) + (\text{ip}_1^{[c..c+1]}(x, d+1) - \text{ip}_1^{[c..c+1]}(x, d)) \cdot (y-d) \\ &= \text{ip}_1^{[c..c+1][d..d+1]}(x, y). \end{aligned} \quad (3.19)$$

So dually to **IP<sub>up</sub>**, the set  $\mathcal{L}$  contains the lemma

$$s > 0 \wedge t \geq d \implies \text{exp}(s, t) \geq \text{ip}_1^{[c..c+1][d..d+1]}(s, t) \quad (\text{IP}_{\text{low}})$$

which is valid due to (3.19) and rules out any spurious counterexample with  $\llbracket \text{exp} \rrbracket(c, d) < c^d$ , as  $\text{ip}_1^{[c..c+1][d..d+1]}(c, d) = c^d$ .

**Lemma 3.21** Let  $c, d \in \mathbb{N}_{>0}$ . Then **IP<sub>low</sub>** is EIA-valid.

*Example 3.22 (Interpolation, Lower Bounds)* Let  $\llbracket \text{exp}(s, t) \rrbracket = \llbracket \text{exp} \rrbracket(3, 9) < 3^9$ , i.e., we have  $c = 3$ , and  $d = 9$ . Then

$$\begin{aligned} \text{ip}_1^{[3..4]}(x, 9) &= 3^9 + (4^9 - 3^9) \cdot (x-3) = 19683 + 242461 \cdot (x-3) \\ \text{ip}_1^{[3..4]}(x, 10) &= 3^{10} + (4^{10} - 3^{10}) \cdot (x-3) = 59049 + 989527 \cdot (x-3) \\ \text{ip}_1^{[3..4][9..10]}(x, y) &= \text{ip}_1^{[3..4]}(x, 9) + (\text{ip}_1^{[3..4]}(x, 10) - \text{ip}_1^{[3..4]}(x, 9)) \cdot (y-9) \end{aligned}$$

and thus we obtain the lemma

$$s > 0 \wedge t \geq 9 \implies \text{exp}(s, t) \geq 747066 \cdot s \cdot t - 6481133 \cdot s - 2201832 \cdot t + 19108788.$$

It is violated by our spurious counterexample, as we have

$$\llbracket 747066 \cdot s \cdot t - 6481133 \cdot s - 2201832 \cdot t + 19108788 \rrbracket = 3^9 > \llbracket \text{exp} \rrbracket(3, 9).$$

**Algorithm 2:** CEGAR for EIA with Lazy Lemma Generation

```

Input: a  $\Sigma_{\text{Int}}^{\text{exp}}$ -formula  $\varphi$ 
// Preprocessing
1 do
2    $\varphi' \leftarrow \varphi$ 
3    $\varphi \leftarrow \text{FOLDCONSTANTS}(\varphi)$ 
4    $\varphi \leftarrow \text{REWRITE}(\varphi)$ 
5 while  $\varphi \neq \varphi'$ 
  // Refinement Loop
6 while there is an UFNIA-model  $\mathbf{A}$  of  $\varphi$ 
7   if  $\mathbf{A}$  is a spurious counterexample then
8      $\mathcal{L} \leftarrow \emptyset$ 
9     for  $k \in \{\text{Negativity}, \text{Monotonicity}, \text{Bounding}\}$ 
10       $\mathcal{L} \leftarrow \{\psi \in \text{COMPUTELEMNAS}(\varphi, k) \mid \mathbf{A} \not\models \psi\}$ 
11      if  $\mathcal{L} \neq \emptyset$  then break
12    if  $\mathcal{L} = \emptyset$  then
13      for  $k \in \{\text{Prime}, \text{Induction}, \text{Interpolation}\}$ 
14         $\mathcal{L} \leftarrow \mathcal{L} \cup \{\psi \in \text{COMPUTELEMNAS}(\varphi, k) \mid \mathbf{A} \not\models \psi\}$ 
15     $\varphi \leftarrow \varphi \wedge \bigwedge \mathcal{L}$ 
16  else return sat
17 return unsat

```

## 3.3 Lazy Lemma Generation

In practice, it is not necessary to compute the entire set of lemmas  $\mathcal{L}$ . Instead, we can stop as soon as  $\mathcal{L}$  contains a single lemma which is violated by the current spurious counterexample. However, such a strategy would result in a quite fragile implementation, as its behavior would heavily depend on the order in which lemmas are computed, which in turn depends on low-level details like the order of iteration over sets, etc. So instead, we use the following precedence on our six kinds of lemmas (where  $A \approx B$  means that  $A$  and  $B$  have the same precedence):

negativity  $\succ$  monotonicity  $\succ$  bounding  $\succ$  prime  $\approx$  induction  $\approx$  interpolation

Then we compute all lemmas of the same kind, starting with negativity lemmas, and we only proceed with kinds of lower precedence if none of the lemmas computed so far is violated by the current spurious counterexample. The motivation for the order above is as follows: First, we prefer negativity, monotonicity, and bounding lemmas, as there are only finitely many of them for a finite set of relevant terms. In contrast, in principle there could be infinitely many prime, induction, and interpolation lemmas, so they all get the same, lowest precedence. The reason is that the natural numbers  $c, d, c^-, c^+, d^-, d^+$  that occur in these lemmas depend on the current UFNIA-model  $\mathbf{A}$ . Negativity lemmas obtain the highest precedence, as other kinds of lemmas depend on them for restricting  $\text{exp}(s, t)$  in the case that  $s$  is negative. Moreover, we prefer monotonicity lemmas over bounding lemmas, as monotonicity lemmas are linear (if the arguments of  $\text{exp}$  are linear), whereas **BND5** may be non-linear.

The improved version of Alg. 1 is shown in Alg. 2. Note that the first inner loop in Line 9 breaks as soon as a lemma that is violated by the current model  $\mathbf{A}$  has been found, i.e., as soon as  $\mathcal{L}$  is non-empty. Then due to the check in Line 12, the second inner loop in Line 13 is only executed if the first inner loop failed to

eliminate the current spurious counterexample. In contrast to the first inner loop, the second one in Line 13 does not break, as prime, induction, and interpolation lemmas have the same precedence.

*Example 3.23 (Leading Example Finished)* We now finish our leading example (†). Among others, our implementation generates the negativity lemmas (3.4) and (3.5), the monotonicity lemma (3.6), and the bounding lemmas (3.7) and (3.8). These lemmas suffice to prove unsatisfiability for the case  $x \geq 3$  (where the case  $y \in [-2..2]$  is trivial). If  $x \geq 3$  and  $y \geq 3$ , we have:

$$y \geq 3 \stackrel{(3.8)}{\curvearrowright} \exp(y, y) \geq y^2 + 2 \stackrel{(3.6)}{\curvearrowright} \exp(x, \exp(y, y)) > \exp(x, y^2) \stackrel{(\dagger)}{\curvearrowright} \text{false}$$

If  $x \geq 3$  and  $y \leq -3$ , we have

$$\begin{aligned} y \leq -3 &\stackrel{(3.4)}{\curvearrowright} \exp(y, y) = 0 \stackrel{(3.7)}{\curvearrowright} \exp(x, \exp(y, y)) = 1 \quad \text{and} \\ x \geq 3 \wedge y \leq -3 &\stackrel{(3.5)}{\curvearrowright} \exp(x, -y^2) = 0 \end{aligned}$$

which, together, contradict  $\exp(x, -y^2) = \exp(x, \exp(y, y))$  and thus also (†). For the case where  $x \leq -3$  and  $|y| \geq 3$ , unsatisfiability can be shown in a similar way, and the case  $x \in [-2..2]$  is again trivial. For the overall proof, 7 negativity lemmas, 2 monotonicity lemmas, and 5 bounding lemmas are used.<sup>17</sup>

While our leading example can be solved without interpolation lemmas, in general, interpolation lemmas are a crucial ingredient of our approach.

*Example 3.24* Consider the unsatisfiable formula

$$\exp(x, y) \neq 19683 \wedge x = 3 \wedge y = 9.$$

To solve it, the interpolation lemmas from Ex. 3.20 and Ex. 3.22 are required. The lemma from Ex. 3.20 is

$$s \in [1, 3] \wedge t \in [1, 9] \implies \exp(s, t) \leq s + \frac{1 + 9841 \cdot (s - 1) - s}{8} \cdot (t - 1),$$

i.e., it implies  $\exp(3, 9) \leq 19683$ . The lemma from Ex. 3.22 is

$$s > 0 \wedge t \geq 9 \implies \exp(s, t) \geq 747066 \cdot s \cdot t - 6481133 \cdot s - 2201832 \cdot t + 19108788,$$

i.e., it implies  $\exp(3, 9) \geq 19683$ .<sup>18</sup>

Our main soundness theorem follows from the soundness of our preprocessings (Lemma 3.4) and the fact that all of our lemmas are EIA-valid (Lemmas 3.8, 3.10, 3.12, 3.14, 3.16, 3.19, and 3.21).

**Theorem 3.25 (Soundness of Alg. 2)** If Alg. 2 returns **sat**, then  $\varphi$  is satisfiable in EIA. If Alg. 2 returns **unsat**, then  $\varphi$  is unsatisfiable in EIA.

<sup>17</sup> To obtain *all* lemmas that are generated by our implementation, the flag `--log` can be used, and to only obtain those lemmas that are used in the final proof of unsatisfiability, our implementation supports the flag `--get-lemmas`.

<sup>18</sup> Note that without interpolation lemmas, Alg. 2 may fail to eliminate a spurious counterexample, as Thm. 3.26 does not hold without interpolation lemmas. In such cases, our implementation returns **unknown**.

Another important property of Alg. 2 is that it can eliminate *any* spurious counterexample, and hence it makes progress in every iteration.

**Theorem 3.26 (Progress Theorem)** If  $\mathbf{A}$  is a spurious counterexample and  $\mathcal{L}$  is computed as in Alg. 2, then

$$\mathbf{A} \not\models \bigwedge \mathcal{L}.$$

Despite Theorems 3.25 and 3.26, EIA is of course undecidable, and hence Alg. 2 is incomplete. For example, it does not terminate for the input formula

$$x \geq 2 \wedge \exp(2, x) + 1 = \exp(3, x). \quad (3.20)$$

The reason is that the only lemmas that provide information about the relation between different  $\exp$ -terms like  $\exp(2, x)$  and  $\exp(3, x)$  are monotonicity lemmas, but they can only be used to deduce  $\exp(2, x) < \exp(3, x)$ , which does not imply  $\exp(2, x) + 1 \neq \exp(3, x)$ . Thus, Alg. 2 would refine the formula (3.20) indefinitely.

Thm. 3.26 can be proved by only considering negativity, bounding, and interpolation lemmas. Nevertheless, monotonicity, prime, and induction lemmas are important as well, even though they are not required to prove Thm. 3.26. The reason is that *all* (usually infinitely many) spurious counterexamples must be eliminated to prove **unsat**. For instance, reconsider Ex. 3.23, where the monotonicity lemma (3.6) eliminates infinitely many spurious counterexamples with  $\llbracket \exp(x, \exp(y, y)) \rrbracket \leq \llbracket \exp(x, y^2) \rrbracket$ . In contrast, Thm. 3.26 only guarantees that every single spurious counterexample can be eliminated. Consequently, our implementation does not terminate on our leading example if monotonicity lemmas are disabled. Similarly, our implementation fails to solve Ex. 3.13 and Ex. 3.15 without prime and induction lemmas, respectively.

## 4 Phasing

When investigating the behavior of our implementation for satisfiable examples where it failed to find a solution, we frequently observed the following problem: Even though a “small” solution exists where the exponents (i.e., the second arguments of the occurring  $\exp$ -terms) have small values, the underlying SMT solver returned candidate models with exponents whose values were quite large. Then interpolation lemmas with huge coefficients were generated, as these coefficients grow exponentially w.r.t. the second argument of  $\exp$ . Afterwards, the underlying solver took too long when searching for a new candidate model.

To counter this phenomenon, we developed *phasing*, a technique to prefer “small” over “large” solutions, see Alg. 3. The idea of phasing is to alternate between a **sat-phase** and an **unsat-phase**.<sup>19</sup> In the **sat-phase**, the goal is to find small candidate models. To this end, we impose a bound on the absolute values of the exponents to block large models (Line 9). If we fail to find a candidate model in the **sat-phase**, then we switch to the **unsat-phase** (Line 26). In the **unsat-phase**, our goal is to prove unsatisfiability, so we do not impose any bounds on the exponents in Line 10. If we fail to prove unsatisfiability, we discard the found model (unless it can be

<sup>19</sup> In our implementation, we do not impose any resource limits on the underlying solver, neither in the **sat-** nor in the **unsat-phase**.

**Algorithm 3:** CEGAR for EIA with Phasing

```

Input: a  $\Sigma_{\text{Int}}^{\text{exp}}$ -formula  $\varphi$ 
// Preprocessing
1 do
2    $\varphi' \leftarrow \varphi$ 
3    $\varphi \leftarrow \text{FOLDCONSTANTS}(\varphi)$ 
4    $\varphi \leftarrow \text{REWRITE}(\varphi)$ 
5 while  $\varphi \neq \varphi'$ 
6    $\text{phase} \leftarrow \text{sat}$ 
7    $b \leftarrow 1$ 
  // Refinement Loop
8 while  $\top$ 
9   if  $\text{phase} = \text{sat}$  then  $\xi \leftarrow \varphi \wedge \{-2^b \leq t \leq 2^b \mid \text{exp}(s, t) \text{ is relevant}\}$ 
10  else  $\xi \leftarrow \varphi$ 
11  if there is an UFNIA-model  $\mathbf{A}$  of  $\xi$  then
12    if  $\mathbf{A}$  is a spurious counterexample then
13      if  $\text{phase} = \text{unsat}$  then
14         $\text{phase} \leftarrow \text{sat}$ 
15         $b++$ 
16      else
17        for  $k \in \{\text{Negativity}, \text{Monotonicity}, \text{Bounding}\}$ 
18           $\mathcal{L} \leftarrow \{\psi \in \text{COMPUTELEMNAS}(\varphi, k) \mid \mathbf{A} \not\models \psi\}$ 
19          if  $\mathcal{L} \neq \emptyset$  then break
20          if  $\mathcal{L} = \emptyset$  then
21            for  $k \in \{\text{Prime}, \text{Induction}, \text{Interpolation}\}$ 
22               $\mathcal{L} \leftarrow \mathcal{L} \cup \{\psi \in \text{COMPUTELEMNAS}(\varphi, k) \mid \mathbf{A} \not\models \psi\}$ 
23             $\varphi \leftarrow \varphi \wedge \bigwedge \mathcal{L}$ 
24        else return sat
25  else if  $\text{phase} = \text{unsat}$  then return unsat
26  else  $\text{phase} \leftarrow \text{unsat}$ 

```

lifted to an EIA-model) in Line 13 and switch back to the **sat**-phase (Line 14), but with a higher bound (Line 15).

Note that the bound grows exponentially w.r.t. the number of phase changes (Lines 9 and 15).<sup>20</sup> Thus, if there are no small models, then the size of the considered models grows rapidly, so that we can still find a solution quickly.

## 5 Related Work

The most closely related work applies *incremental linearization* to NIA, or to non-linear real arithmetic with transcendental functions (NRAT). Like our approach, incremental linearization is an instance of the CEGAR paradigm: An initial abstraction (where certain predefined functions are considered as uninterpreted functions) is refined via linear lemmas that rule out the current spurious counterexample.

Our approach is inspired by, but differs significantly from the approach for linearization of NRAT by Cimatti et al. [15]. There, non-linear polynomials are linearized as well, whereas we leave the handling of polynomials to the backend solver. Moreover, Cimatti et al. use linear lemmas only, whereas we also use bilinear

<sup>20</sup> The initial value of  $b$ , as well as the base 2 in Line 9 are somewhat arbitrary. Depending on the benchmarks, other values might yield better performance.

lemmas. Furthermore, Cimatti et al. fix the base to Euler’s number  $e$ , whereas we consider a binary version of exponentiation.

The only lemmas of Cimatti et al. [15] that easily carry over to our approach are monotonicity lemmas. Our bounding lemmas are related to the “lower bound” and “zero” lemmas of Cimatti et al., but there,  $\lambda x. e^x$  is trivially bounded by 0. Interpolation lemmas are related to the “tangent” and “secant lemmas” of Cimatti et al.. However, tangent lemmas make use of first derivatives, so they are not expressible with integer arithmetic in our setting, as we have  $\frac{\partial}{\partial y} x^y = x^y \cdot \ln x$ . Secant lemmas are essentially obtained by linear interpolation, so our interpolation lemmas can be seen as a generalization of secant lemmas to binary functions. The preprocessing by rewriting, prime lemmas, or induction lemmas are not considered by Cimatti et al..

Cimatti et al. [14] also applied incremental linearization to NIA. Here, they used similar lemmas as for NRAT [15], so they differ again fundamentally from ours.

Further existing approaches for NRAT are based on interval propagation [18, 29]. As observed by Cimatti et al. [15], interval propagation effectively computes a piecewise *constant* approximation, which is less expressive than our bilinear approximations.

In recent years, a novel approach for NRAT based on the *topological degree test* has been proposed [16, 36]. Its strength is finding irrational solutions more often than other approaches for NRAT. Hence, this line of work is orthogonal to ours.

EIA could also be tackled by combining NRAT techniques with branch-and-bound, but the following example shows that doing so is not promising.

*Example 5.1* Consider the formula  $x = \exp(3, y) \wedge y > 0$ . To handle it with existing solvers, we have to encode it using the natural exponential function:

$$e^z = 3 \wedge x = e^{y \cdot z} \wedge y > 0 \quad (5.1)$$

Here  $x$  and  $y$  range over the integers and  $z$  ranges over the reals. Any model of (5.1) satisfies  $z = \ln 3$ , where  $\ln 3$  is irrational. As finding such models is challenging, the leading tools MathSAT [13] and cvc5 [3] fail for  $e^z = 3$ .

Recently, a variant of our original approach from [26] has been integrated into cvc5 [3], where it is used to solve SMT problems with parametric bitvectors [7]. In their setting, the base of  $\exp$  is fixed to 2, i.e., they consider a special case of our work. Moreover, they do not use interpolation, prime, or induction lemmas, i.e., their solver for exponentials is significantly simpler than ours. Note, however, that their primary goal is to solve problems with parametric bitvectors, and their solver for exponentials is just one component in a sophisticated toolchain for that purpose.

MetiTarski [1] integrates decision procedures for real closed fields and approximations for transcendental functions into the theorem prover Metis [32] to prove theorems about the reals. In a related line of work, iSAT3 [18] has been coupled with SPASS [48]. Clearly, these approaches differ fundamentally from ours.

Recently, the complexity of decidable extensions of linear integer arithmetic with exponentiation has been investigated [6, 12, 34]. Benedikt et al. [6] consider *Semenov arithmetic*, which extends linear integer arithmetic with a function  $\lambda x. c^{|x|}$  for some fixed natural number  $c \geq 2$ . Thus, it is related to EIA with quantifiers,

but without the functions “.”, “div”, and “mod”, and where the first argument of all occurrences of `exp` must be the same constant. In contrast, Chistikov et al. [12] consider only quantifier-free conjunctions of literals from Semenov arithmetic. Adapting such decision procedures to the SMT-LIB semantics of `exp` and integrating them into our approach is an interesting direction for future work. The technique of Chistikov et al. is more efficient than the one of Benedikt et al., but as it can only handle conjunctions of literals, it needs to be implemented as a theory solver, i.e., *within* an SMT solver. In contrast, our approach can be implemented *on top of* an SMT solver, which is easier, as it does not require detailed knowledge of the internals of the SMT solver. Thus, the approach of Benedikt et al. seems to be a better fit for our framework.

Karimov et al. [34] showed that decidability of the existential fragment of an extension of Semenov arithmetic with a second exponential function (where the bases are multiplicatively independent) would lead to mathematical breakthroughs. So as it stands, we cannot hope for a complete solver as soon as we allow exponentiation with different bases.

An alternative way to solve SMT problems with exponentiation is to provide the recursive definition of `exp` as universally quantified axioms. Then SMT solvers with support for quantifiers try to instantiate these axioms suitably in order to derive a conflict. To find suitable instantiations, various heuristics have been proposed [38, 42–45]. However, such approaches are only suitable for proving *unsatisfiability*, and for that purpose, our evaluation shows that our approach outperforms quantifier instantiation techniques, see Sect. 6.4.

## 6 Implementation and Evaluation

We implemented our approach in our novel tool `SwlnE2`, which is based on Z3 4.15.4 [39]. To improve efficiency, `SwlnE2` uses Z3’s incremental solving capabilities.

In contrast to `SwlnE2`, our implementation `SwlnE1` from [26] does not support prime lemmas, induction lemmas, and phasing. While `SwlnE2` directly uses the API of Z3, `SwlnE1` is based on SMT-Switch [37], a library that offers a unified interface for various SMT solvers. By default, `SwlnE1` also uses Z3. For more information on the different versions of `SwlnE`, including precompiled releases for all platforms, we refer to [28].

### 6.1 Other Tools

We compare our implementation with `SwlnE1`, Z3 4.15.4, and `cvc5` 1.3.4 [3]. Note that `SwlnE1` interprets `exp` as  $\llbracket \text{exp} \rrbracket (x, y) = x^{|y|}$ , i.e., the semantics of `exp` for negative exponents differ in `SwlnE1` and `SwlnE2`. However, by construction, our benchmarks do not use negative exponents (see below for details). So the different semantics are irrelevant for their satisfiability.

To compare with Z3, we added the following, universally quantified axioms to each benchmark which specify the semantics of `exp`:

$$\begin{array}{ll}
\forall x \in \mathbb{Z}. & \exp(x, 0) = 1 \\
\forall x, y \in \mathbb{Z}. & y > 0 \implies \exp(x, y) = x \cdot \exp(x, y - 1) \\
\forall y \in \mathbb{Z}. & \exp(1, y) = 1 \\
\forall y \in \mathbb{Z}. & \text{divisible}_2(y) \implies \exp(-1, y) = 1 \\
\forall y \in \mathbb{Z}. & \neg \text{divisible}_2(y) \implies \exp(-1, y) = -1 \\
\forall x, y \in \mathbb{Z}. & \text{abs}(x) \neq 1 \wedge y < 0 \implies \exp(x, y) = 0
\end{array}$$

Here, the first two assertions define the semantics of  $\exp$  for all non-negative values of  $y$ , the next three assertions define its semantics for the case  $\text{abs}(x) = 1$ , and the last assertion covers the missing case  $\text{abs}(x) \neq 1 \wedge y < 0$ .

Note that SMT solvers use quantifier instantiation to derive contradictions, but adding such quantified assertions prevents them from proving satisfiability. Thus, Z3 can only prove unsatisfiability in our benchmarks.

To compare with `cvc5`, we consider two configurations: For one of them (called “`cvc5` with axioms” in the sequel), we added the universally quantified axioms provided above, analogously to our comparison with Z3. Thus, `cvc5` with axioms can only prove unsatisfiability, too. For the second one (called “`cvc5 pow2`”), we only used benchmarks where all occurrences of  $\exp$  have the form  $\exp(2, \dots)$ . Such occurrences can be replaced by the function `pow2`, which is supported by `cvc5`.

## 6.2 Benchmarks

To evaluate our approach, we synthesized a large collection of EIA problems from verification benchmarks for safety, termination, and complexity analysis. More precisely, we ran our verification tool LoAT [22] on the benchmarks for *linear Constrained Horn Clauses (CHCs)* with *linear integer arithmetic* from the *CHC Competitions* 2022 and 2023 as well as on the benchmarks for *Termination* and *Complexity of Integer Transition Systems* from the *Termination Problems Database (TPDB)* [46], the benchmark set of the *Termination and Complexity Competition* [30], and extracted all SMT problems with exponentiation that LoAT created while analyzing these benchmarks. Afterwards, we removed duplicates.

The resulting benchmark set consists of 4627 SMT problems (called *LoAT Problems* below), which are available online [27]:

- 669 problems that resulted from the benchmarks of the CHC Competition ’22 (called *CHC Comp ’22 Problems* below)
- 158 problems that resulted from the benchmarks of the CHC Competition ’23 (*CHC Comp ’23 Problems*)
- 3146 problems that resulted from the complexity benchmarks of the TPDB (*Complexity Problems*)
- 654 problems that resulted from the termination benchmarks of the TPDB (*Termination Problems*)

By construction, the second argument of  $\exp$  always corresponds to the number of iterations of a loop in these benchmarks, and thus the exponents are natural numbers.

Unfortunately, these sets do not contain challenging unsatisfiable benchmarks: As discussed in Sect. 6.3, most unsatisfiable instances can be solved without

reasoning about exponentials at all. More precisely, these benchmarks are not just unsatisfiable in EIA (where the interpretation of  $\exp$  is fixed), but they are also unsatisfiable in UFNIA, where  $\exp$  is an uninterpreted function. Thus, they can be solved by off-the-shelf SMT solvers without support for exponentiation. To obtain challenging unsatisfiable examples, we generated one more set of benchmarks by validating the results of the recurrence solver PURRS [2]. More precisely, we considered all recurrence relations from PURRS's test suite<sup>21</sup> where both the recurrence relation and its expected solution can be expressed with polynomials and exponentials. Consider such a recurrence relation

$$f(n) = t \tag{6.1}$$

where  $f$  is the inductively defined function, i.e., the term  $t$  contains subterms of the form  $f(n - c)$ ,  $c \in \mathbb{N}_{>0}$ . Let  $k$  be the maximal natural number such that  $f(n - k)$  is a subterm of  $t$ . As an example, consider the recurrence relation (3.10) in Ex. 3.15 where  $t$  is  $2 \cdot f(n - 1) + 2^n$  and thus,  $k = 1$ . Moreover, let  $s$  be the expected solution for (6.1) given in the test suite. For such expected solutions, we always have  $n \in \mathcal{V}(s) \subseteq \mathcal{V}(t)$ , and for all subterms  $f(q)$  of  $s$ , we have  $q \in \{0, \dots, k - 1\}$ . Here,  $\mathcal{V}(s)$  and  $\mathcal{V}(t)$  denote the sets of variables occurring in  $s$  and  $t$ , respectively. For the solution (3.11) in Ex. 3.15, we have  $s = (f(0) + n) \cdot 2^n$ , i.e., here the only  $f$ -subterm of  $s$  is  $f(0)$ . To construct new benchmarks, we considered the SMT problem

$$n \geq k \wedge s_{\exp} \neq t_{\exp}[f(n - i)/s_{\exp}[n/n - i] \mid 1 \leq i \leq k] \tag{6.2}$$

where  $s_{\exp}$  and  $t_{\exp}$  result from  $s$  and  $t$  by replacing all exponentials (i.e., all subterms of the form  $p^q$  where  $q \notin \mathbb{Z}$ ) with  $\exp(p, q)$ . Moreover,  $s_{\exp}[n/n - i]$  results from  $s_{\exp}$  by replacing  $n$  with  $n - i$  and  $t_{\exp}[f(n - i)/s_{\exp}[n/n - i] \mid 1 \leq i \leq k]$  results from  $t_{\exp}$  by replacing each subterm  $f(n - i)$  with  $s_{\exp}[n/n - i]$ , for all  $1 \leq i \leq k$ . In this way, we obtained the formula (3.12) in Ex. 3.15. If (6.2) is unsatisfiable, then the given solution is correct. More precisely, let  $\vec{x} = \mathcal{V}(s) \setminus \{n\}$ . Then the unsatisfiability of (6.2) implies that the equation (6.1) holds for all  $n \in \mathbb{N}$ , all  $\vec{x} \in \mathbb{Z}^{|\vec{x}|}$ , and all  $c_0, \dots, c_{k-1} \in \mathbb{Z}$  when interpreting  $f$  as

$$n \mapsto \begin{cases} c_n & \text{if } n < k \\ s[f(i)/c_i \mid 0 \leq i < k] & \text{otherwise.} \end{cases}$$

In this way, we obtained 37 hard unsatisfiable benchmarks (called PURRS *Problems* below), which are also available online [27].

In these benchmarks, the second argument of  $\exp$  is always a function in  $n$  which is non-negative for all  $n \geq k$ , i.e., the exponents are again natural numbers.

### 6.3 Evaluation on the LoAT Problems

We first report on our experiments on the LoAT Problems. To evaluate the impact of the different components of our approach, we tested with configurations where we disabled rewriting, negativity lemmas, monotonicity lemmas, bounding lemmas, prime lemmas, induction lemmas, interpolation lemmas, phasing, or all of these techniques.

<sup>21</sup> <https://github.com/aprove-developers/LoAT-purrs/blob/master/tests/recurrences>

All experiments were performed on [CLAIX-2023-HPC nodes](#) of the RWTH University High Performance Computing Cluster.<sup>22</sup> Each such node has 96 cores, and each benchmark had exclusive access to one core and 10560 MiB ( $\approx 11$ GB) of memory, but several benchmarks were run in parallel on the same node. The wall clock timeout was 10 s per example. We chose a small timeout, as LoAT usually has to discharge many SMT problems to solve a single verification task. So in our setting, each individual SMT problem should be solved quickly.

The results for the different groups of benchmarks can be seen in Fig. 6.1 – 6.4, and Fig. 6.5 summarizes the results for all 4627 LoAT Problems. Here, we provide the number of timeouts in parentheses in the but-last column (i.e., “**unsolved**” refers to all examples with timeouts or with the result **unknown**). So for example, Z3 failed to solve 296 CHC Comp ’22 Problems, and in 285 of these cases, it timed out. The last column is only relevant for `cvc5 pow2`, where it denotes the number of problems that cannot be analyzed with this configuration, as they contain occurrences of `exp` that cannot be replaced with `pow2`.

All but two of the 4627 benchmarks can be solved by `SwInE2`. One unsolved instance cannot be solved by any configuration, and for the other one, satisfiability can be proven by `SwInE1`, `SwInE2` without rewriting, and `SwInE2` without phasing. Most configurations of `SwInE2` as well as `SwInE1` can solve (almost<sup>23</sup>) all CHC Comp and Termination Problems. The only exceptions are the configurations without bounding or interpolation lemmas, which clearly shows their importance.

The Complexity Problems (Fig. 6.3) are by far the hardest, and thus also the most interesting. Here, `SwInE2` clearly outperforms `SwInE1` (2 vs. 59 unsolved instances). Moreover, Fig. 6.3 shows that all components of our approach except for monotonicity lemmas, prime lemmas, and induction lemmas are crucial for the performance of `SwInE2` on this benchmark set.

The fact that monotonicity lemmas are of little importance here is not surprising: Monotonicity lemmas were already of little use when Z3 was used as the backend solver in the evaluation of `SwInE1` [26]. However, these lemmas significantly improved the performance of `SwInE1` with `cvc5` [3] as the backend solver. Hence, the usefulness of monotonicity lemmas depends on the details of the underlying solver. Moreover, recall that our leading example cannot be solved without monotonicity lemmas, see Ex. 3.23.

It is also not surprising that induction lemmas do not contribute much to the performance of `SwInE2` on this benchmark set, as their main purpose is proving unsatisfiability, see Sect. 6.4.

However, it is slightly disappointing that prime lemmas are not needed for the LoAT problems. Nevertheless, we believe that prime factorizations are of such fundamental relevance that prime lemmas are likely to be useful for benchmarks from other sources in the future.

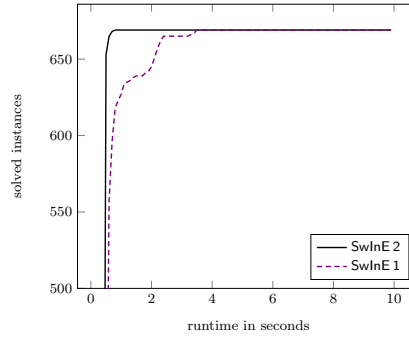
In the configuration “no lemmas”, we disabled all components of our approach, i.e., besides all lemmas, we also disabled rewriting and phasing. Thus, this approach boils down to calling Z3 directly and treating `exp` like an uninterpreted function. Thus, the high number of unsatisfiability proofs for this configuration shows that the unsatisfiable instances in this benchmark collection are mostly trivial in the sense that they do not require any reasoning about `exp`, as explained in Sect. 6.2.

<sup>22</sup> <https://help.itc.rwth-aachen.de/service/rhr4fjjuttff>

<sup>23</sup> Without phasing, `SwInE2` times out on one CHC Comp benchmark.

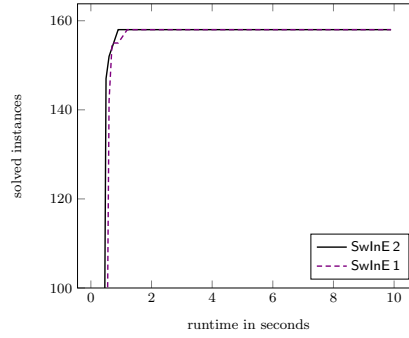
| solver  | configuration    | sat | unsat | unsolved (⊙) | n/a |
|---------|------------------|-----|-------|--------------|-----|
| SwInE 2 | default          | 296 | 373   | 0            | 0   |
| SwInE 1 |                  | 296 | 373   | 0            | 0   |
| Z3      | axioms           | n/a | 373   | 296 (285)    | 0   |
| cvc5    | pow2             | 142 | 165   | 0            | 362 |
|         | axioms           | n/a | 372   | 297 (222)    | 0   |
| SwInE 2 | no rewriting     | 296 | 373   | 0            | 0   |
|         | no negativity    | 296 | 373   | 0            | 0   |
|         | no monotonicity  | 296 | 373   | 0            | 0   |
|         | no bounding      | 296 | 373   | 0            | 0   |
|         | no prime         | 296 | 373   | 0            | 0   |
|         | no induction     | 296 | 373   | 0            | 0   |
|         | no interpolation | 187 | 373   | 110 (0)      | 0   |
|         | no phasing       | 296 | 372   | 1 (1)        | 0   |
|         | no lemmas        | 49  | 364   | 256 (0)      | 0   |

Fig. 6.1: CHC Comp '22



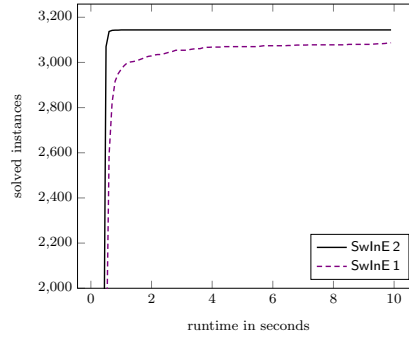
| solver  | configuration    | sat | unsat | unsolved (⊙) | n/a |
|---------|------------------|-----|-------|--------------|-----|
| SwInE 2 | default          | 87  | 71    | 0            | 0   |
| SwInE 1 |                  | 87  | 71    | 0            | 0   |
| Z3      | axioms           | n/a | 71    | 87 (87)      | 0   |
| cvc5    | pow2             | 9   | 3     | 0            | 146 |
|         | axioms           | n/a | 71    | 87 (42)      | 0   |
| SwInE 2 | no rewriting     | 87  | 71    | 0            | 0   |
|         | no negativity    | 87  | 71    | 0            | 0   |
|         | no monotonicity  | 87  | 71    | 0            | 0   |
|         | no bounding      | 87  | 71    | 0            | 0   |
|         | no prime         | 87  | 71    | 0            | 0   |
|         | no induction     | 87  | 71    | 0            | 0   |
|         | no interpolation | 8   | 71    | 79 (0)       | 0   |
|         | no phasing       | 87  | 71    | 0            | 0   |
|         | no lemmas        | 4   | 61    | 93 (0)       | 0   |

Fig. 6.2: CHC Comp '23



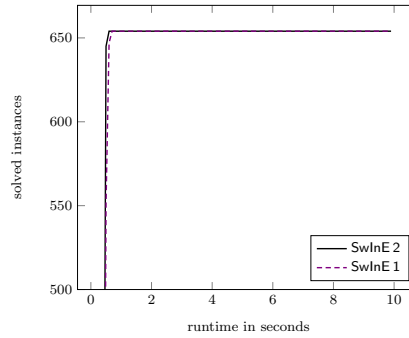
| solver  | configuration    | sat  | unsat | unsolved (⊙) | n/a |
|---------|------------------|------|-------|--------------|-----|
| SwInE 2 | default          | 1355 | 1789  | 2 (2)        | 0   |
| SwInE 1 |                  | 1298 | 1789  | 59 (53)      | 0   |
| Z3      | axioms           | n/a  | 1753  | 1393 (1351)  | 0   |
| cvc5    | pow2             | 872  | 1324  | 1 (0)        | 949 |
|         | axioms           | n/a  | 1753  | 1393 (1370)  | 0   |
| SwInE 2 | no rewriting     | 1319 | 1789  | 38 (38)      | 0   |
|         | no negativity    | 1278 | 1789  | 79 (2)       | 0   |
|         | no monotonicity  | 1355 | 1789  | 2 (2)        | 0   |
|         | no bounding      | 932  | 1789  | 425 (1)      | 0   |
|         | no prime         | 1355 | 1789  | 2 (2)        | 0   |
|         | no induction     | 1355 | 1789  | 2 (2)        | 0   |
|         | no interpolation | 1324 | 1788  | 34 (1)       | 0   |
|         | no phasing       | 1343 | 1789  | 14 (10)      | 0   |
|         | no lemmas        | 109  | 1733  | 1304 (1)     | 0   |

Fig. 6.3: Complexity



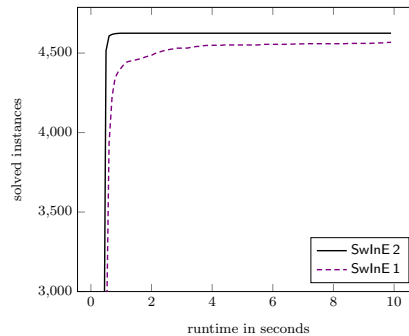
| solver  | configuration    | sat | unsat | unsolved (⊙) | n/a |
|---------|------------------|-----|-------|--------------|-----|
| SwInE 2 | default          | 223 | 431   | 0            | 0   |
| SwInE 1 |                  | 223 | 431   | 0            | 0   |
| Z3      | axioms           | n/a | 428   | 226 (219)    | 0   |
| cvc5    | pow2             | 212 | 425   | 0            | 17  |
|         | axioms           | n/a | 428   | 226 (224)    | 0   |
| SwInE 2 | no rewriting     | 223 | 431   | 0            | 0   |
|         | no negativity    | 223 | 431   | 0            | 0   |
|         | no monotonicity  | 223 | 431   | 0            | 0   |
|         | no bounding      | 196 | 429   | 29 (0)       | 0   |
|         | no prime         | 223 | 431   | 0            | 0   |
|         | no induction     | 223 | 431   | 0            | 0   |
|         | no interpolation | 63  | 429   | 162 (0)      | 0   |
|         | no phasing       | 223 | 431   | 0            | 0   |
|         | no lemmas        | 19  | 431   | 207 (0)      | 0   |

Fig. 6.4: Termination



| solver  | configuration    | sat  | unsat | unsolved (⊙) | n/a  |
|---------|------------------|------|-------|--------------|------|
| SwInE 2 | default          | 1961 | 2664  | 2 (2)        | 0    |
| SwInE 1 |                  | 1904 | 2664  | 59 (53)      | 0    |
| Z3      | axioms           | n/a  | 2625  | 2202 (1942)  | 0    |
| cvc5    | pow2             | 1235 | 1917  | 1 (0)        | 1474 |
|         | axioms           | n/a  | 2624  | 2003 (1858)  | 0    |
| SwInE 2 | no rewriting     | 1925 | 2664  | 38 (38)      | 0    |
|         | no negativity    | 1884 | 2664  | 79 (2)       | 0    |
|         | no monotonicity  | 1961 | 2664  | 2 (2)        | 0    |
|         | no bounding      | 454  | 2662  | 454 (1)      | 0    |
|         | no prime         | 1961 | 2664  | 2 (2)        | 0    |
|         | no induction     | 1961 | 2664  | 2 (2)        | 0    |
|         | no interpolation | 1582 | 2664  | 385 (1)      | 0    |
|         | no phasing       | 1949 | 2663  | 15 (11)      | 0    |
|         | no lemmas        | 181  | 2586  | 1860 (1)     | 0    |

Fig. 6.5: All LoAT Problems



Consequently, it is also not surprising that `cvc5` and `Z3` perform very well on the unsatisfiable instances with additional axioms for `exp`, where the axioms allow them to prove unsatisfiability in even more cases.

The configuration `cvc5 pow2` also performs very well: It solves all but one benchmark where the base of all exponentials is 2. However, all of these instances are also solved by `SwInE2` (including the one where `cvc5 pow2` fails).

The runtimes can be seen in the plots on the right-hand sides of Fig. 6.1 – 6.5. Most instances can be solved in a fraction of a second, as desired for our use case. On the CHC Comp '23 and the Termination Problems, `SwInE2` and `SwInE1` are equally efficient. On the CHC Comp '22 and the Complexity Problems, `SwInE2` is slightly faster than `SwInE1`. We refer to [27] for more details.

#### 6.4 Evaluation on the PURRS Problems

We continue with the evaluation on the PURRS Problems, where we used a wall clock timeout of 300s. All of them are unsatisfiable (provided that the test cases from PURRS's test suite are correct), and most of these problems require inductive reasoning, so they are particularly suitable to illustrate the usefulness of induction lemmas. We considered three configurations of `SwInE2`: The default configuration (where all features are enabled), a configuration without induction lemmas, and a configuration without prime lemmas.

The results can be seen in Table 6.1. The default configuration of `SwInE2` subsumes all other tools and configurations, and it only fails for 5 instances. Without prime lemmas, `SwInE2` fails to solve one more instance, and without induction lemmas, `SwInE2` is clearly not suitable for this benchmark set. The same is true for `cvc5 pow2`, which also instantiates lemmas based on spurious counterexamples, but does not support induction lemmas.

Regarding the benchmarks that are also solved by `Z3` or `cvc5`, the runtime is usually comparable, with some notable exceptions (e.g., `purrs17` for `SwInE2` and `purrs19` for `cvc5`).

### 7 Conclusion

Inspired by *incremental linearization*, we developed a CEGAR approach to solve EIA problems, i.e., SMT problems with *exponential integer arithmetic*. The core idea of our approach is to regard exponentiation as an uninterpreted function and to eliminate spurious counterexamples, i.e., models that violate the semantics of exponentiation, by generating suitable *lemmas*. Here, the use of *bilinear interpolation* turned out to be crucial for proving satisfiability, both in practice (see our evaluation in Sect. 6) and in theory, as interpolation lemmas are essential for being able to eliminate *any* spurious counterexample (see Thm. 3.26). Finally, we evaluated the implementation of our approach in our novel tool `SwInE2` on thousands of EIA problems that were synthesized from verification tasks using our verification tool `LoAT`, and on challenging unsatisfiable benchmarks that result from the verification of solutions for recurrence relations. Our evaluation shows that `SwInE2` is highly effective for both use cases. In particular, it shows that *induction lemmas* are crucial

| tool    | SwInE 2 |              |          |      | Z3     |       | cvc5   |      |    |       |     |        |
|---------|---------|--------------|----------|------|--------|-------|--------|------|----|-------|-----|--------|
| config  | default | no induction | no prime |      | axioms |       | axioms | pow2 |    |       |     |        |
| purrs01 | ✓       | 0.40         | ⊖        | —    | ✓      | 0.47  | ✓      | 0.37 | ✓  | 0.60  | ✗   | 100.94 |
| purrs02 | ✓       | 0.35         | ⊖        | —    | ✓      | 0.35  | ✓      | 0.38 | ⊖  | —     | ✗   | 145.12 |
| purrs03 | ⊖       | —            | ⊖        | —    | ⊖      | —     | ⊖      | —    | ⊖  | —     | n/a | —      |
| purrs04 | ✓       | 0.38         | ⊖        | —    | ✓      | 0.40  | ✓      | 0.36 | ✓  | 0.41  | n/a | —      |
| purrs05 | ✓       | 0.51         | ⊖        | —    | ✓      | 0.36  | ✓      | 0.33 | ✓  | 0.38  | n/a | —      |
| purrs06 | ✓       | 0.35         | ⊖        | —    | ✓      | 0.35  | ✓      | 0.39 | ✓  | 0.38  | n/a | —      |
| purrs07 | ✓       | 1.14         | ⊖        | —    | ✓      | 0.82  | ✗      | 4.4  | ⊖  | —     | n/a | —      |
| purrs08 | ✓       | 0.45         | ✓        | 0.50 | ✓      | 0.46  | ✓      | 0.39 | ✓  | 0.70  | n/a | —      |
| purrs09 | ✓       | 0.43         | ⊖        | —    | ✓      | 0.45  | ⊖      | —    | ⊖  | —     | n/a | —      |
| purrs10 | ✓       | 0.46         | ⊖        | —    | ✓      | 0.41  | ⊖      | —    | ⊖  | —     | n/a | —      |
| purrs11 | ✓       | 0.36         | ⊖        | —    | ✓      | 0.41  | ✓      | 0.38 | ⊖  | —     | ⊖   | —      |
| purrs12 | ⊖       | —            | ⊖        | —    | ⊖      | —     | ⊖      | —    | ⊖  | —     | n/a | —      |
| purrs13 | ✓       | 1.2          | ⊖        | —    | ✓      | 0.47  | ✓      | 0.45 | ⊖  | —     | n/a | —      |
| purrs14 | ✓       | 0.35         | ✓        | 0.36 | ✓      | 0.34  | ✓      | 0.39 | ✓  | 0.40  | n/a | —      |
| purrs15 | ✓       | 0.54         | ⊖        | —    | ✓      | 0.37  | ⊖      | —    | ⊖  | —     | n/a | —      |
| purrs16 | ⊖       | —            | ⊖        | —    | ⊖      | —     | ⊖      | —    | ⊖  | —     | n/a | —      |
| purrs17 | ✓       | 105.39       | ⊖        | —    | ✓      | 1.32  | ⊖      | —    | ✓  | 1.48  | n/a | —      |
| purrs18 | ✓       | 0.62         | ⊖        | —    | ✓      | 3.17  | ✓      | 0.38 | ✓  | 4.24  | n/a | —      |
| purrs19 | ✓       | 0.63         | ⊖        | —    | ✓      | 0.47  | ✓      | 0.37 | ✓  | 70.19 | n/a | —      |
| purrs20 | ✓       | 0.44         | ⊖        | —    | ✓      | 0.46  | ✓      | 0.38 | ✓  | 0.42  | n/a | —      |
| purrs21 | ✓       | 0.34         | ⊖        | —    | ✓      | 0.35  | ✓      | 0.36 | ✓  | 0.38  | n/a | —      |
| purrs22 | ✓       | 0.36         | ⊖        | —    | ✓      | 0.33  | ✓      | 0.37 | ✓  | 0.37  | ✗   | 5.35   |
| purrs23 | ✓       | 0.37         | ⊖        | —    | ✓      | 0.42  | ✓      | 0.39 | ✓  | 7.5   | ✗   | 249.6  |
| purrs24 | ✓       | 0.43         | ⊖        | —    | ✓      | 0.43  | ✓      | 0.38 | ✓  | 0.39  | n/a | —      |
| purrs25 | ✓       | 0.36         | ⊖        | —    | ✓      | 0.34  | ✓      | 0.37 | ✓  | 4.49  | ✗   | 2.33   |
| purrs26 | ✓       | 0.36         | ⊖        | —    | ✓      | 0.35  | ✓      | 0.36 | ✓  | 0.36  | ✗   | 14.68  |
| purrs27 | ✓       | 0.33         | ⊖        | —    | ✓      | 0.31  | ✓      | 0.36 | ✓  | 0.35  | ✗   | 3.25   |
| purrs28 | ✓       | 0.36         | ⊖        | —    | ✓      | 0.33  | ✓      | 0.38 | ✓  | 0.36  | ✗   | 4.71   |
| purrs29 | ✓       | 0.37         | ⊖        | —    | ✓      | 0.37  | ✓      | 0.40 | ⊖  | —     | ⊖   | —      |
| purrs30 | ✓       | 13.99        | ⊖        | —    | ✓      | 14.34 | ✓      | 1.74 | ⊖  | —     | n/a | —      |
| purrs31 | ⊖       | —            | ⊖        | —    | ⊖      | —     | ⊖      | —    | ✗  | 0.39  | ✗   | 0.38   |
| purrs32 | ⊖       | —            | ⊖        | —    | ⊖      | —     | ⊖      | —    | ⊖  | —     | n/a | —      |
| purrs33 | ✓       | 0.76         | ⊖        | —    | ✓      | 0.63  | ✓      | 0.42 | ✓  | 0.53  | n/a | —      |
| purrs34 | ✓       | 0.34         | ✓        | 0.35 | ✓      | 0.33  | ✓      | 0.37 | ✓  | 0.38  | n/a | —      |
| purrs35 | ✓       | 8.59         | ⊖        | —    | ⊖      | —     | ⊖      | —    | ⊖  | —     | n/a | —      |
| purrs36 | ✓       | 2.5          | ⊖        | —    | ✓      | 4.47  | ⊖      | —    | ⊖  | —     | n/a | —      |
| purrs37 | ✓       | 2.34         | ⊖        | —    | ✓      | 2.39  | ✓      | 1.16 | ✓  | 30.70 | n/a | —      |
| solved  | 32      |              | 3        |      | 31     |       | 25     |      | 21 |       | 0   |        |

✓: unsat, ⊖: timeout, ✗: unknown

Table 6.1: PURRS Problems

for proving unsatisfiability, and that our new *phasing* technique allows our novel implementation SwInE 2 to solve more satisfiable instances.

Consequently, we integrated SwInE 2 into our verification tool LoAT, where it now serves as the default solver for all techniques that require reasoning about exponentials [23, 25].

With SwInE 2, we provide an SMT-LIB compliant open-source solver for EIA [28]. In this way, we hope to attract users with applications that give rise to challenging benchmarks, and we hope that other solvers with support for integer exponentiation will follow.

**Acknowledgements** This work has been funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - 235950644 (Project GI 274/6-2).

*Personal Remarks from Florian Frohn:* I first met Christoph at my first conference, IJCAR '16, where he and his group presented two stunning papers: One about a verified SAT solver [8] (which won the best paper award), and one about the cube test for linear integer arithmetic [11] (which quickly became a standard technique in SMT, and probably would have deserved the best paper award as well). Three years later, I had the honor to become a postdoc in his group. During our first meeting, we discussed potential topics for collaborations, together with Jürgen and Thomas Sturm. One of these topics was SMT with integer exponentiation. Unfortunately, this work did not materialize during my time in his group. Thus, I am happy about the chance to contribute it to Christoph's Festschrift instead.

*Personal Remarks from Jürgen Giesl:* I have known Christoph already for a long time, and I am always impressed by both the depth and the breadth of his research, which I had the privilege to study more deeply during my time on the Scientific Advisory Board of the Max-Planck-Institute in Saarbrücken. I greatly admire Christoph's energy and enthusiasm, also when it comes to the organization and strategic aspects of research. I learned a lot from him during his time as president of CADE, and as his successor in this position, I am always grateful for his advice. I am really delighted to contribute our paper to Christoph's Festschrift.

## References

1. Akbargpour B, Paulson LC (2010) MetiTarski: An automatic theorem prover for real-valued special functions. *J Autom Reason* 44(3):175–205, doi:[10.1007/S10817-009-9149-2](https://doi.org/10.1007/S10817-009-9149-2)
2. Bagnara R, Pescetti A, Zaccagnini A, Zaffanella E (2005) PURRS: Towards computer algebra support for fully automatic worst-case complexity analysis. *CoRR abs/cs/0512056*, doi:[10.48550/arXiv.cs/0512056](https://doi.org/10.48550/arXiv.cs/0512056)
3. Barbosa H, Barrett CW, Brain M, Kremer G, Lachnitt H, Mann M, Mohamed A, Mohamed M, Niemetz A, Nötzli A, Ozdemir A, Preiner M, Reynolds A, Sheng Y, Tinelli C, Zohar Y (2022) cvc5: A versatile and industrial-strength SMT solver. In: *TACAS '22*, LNCS 13243, pp 415–442, doi:[10.1007/978-3-030-99524-9\\_24](https://doi.org/10.1007/978-3-030-99524-9_24)
4. Bardin S, Finkel A, Leroux J, Petrucci L (2008) FAST: Acceleration from theory to practice. *Int J Softw Tools Technol Transf* 10(5):401–424, doi:[10.1007/s10009-008-0064-3](https://doi.org/10.1007/s10009-008-0064-3)
5. Barrett C, Fontaine P, Tinelli C (2016) The Satisfiability Modulo Theories Library (SMT-LIB). <https://www.SMT-LIB.org>. [www.SMT-LIB.org](http://www.SMT-LIB.org)
6. Benedikt M, Chistikov D, Mansutti A (2023) The complexity of Presburger arithmetic with power or powers. In: *ICALP '23*, LIPIcs 261, pp 112:1–112:18, doi:[10.4230/LIPIcs.ICALP.2023.112](https://doi.org/10.4230/LIPIcs.ICALP.2023.112)
7. Berger Z, Zohar Y, Niemetz A, Preiner M, Reynolds A, Barrett CW, Tinelli C (2025) Bit-precise reasoning with parametric bit-vectors. In: *SAT '25*, LIPIcs 341, pp 4:1–4:24, doi:[10.4230/LIPICS.SAT.2025.4](https://doi.org/10.4230/LIPICS.SAT.2025.4)
8. Blanchette JC, Fleury M, Weidenbach C (2016) A verified SAT solver framework with learn, forget, restart, and incrementality. In: *IJCAR '16*, LNCS 9706, pp 25–44, doi:[10.1007/978-3-319-40229-1\\_4](https://doi.org/10.1007/978-3-319-40229-1_4)
9. Bozga M, Iosif R, Konečný F (2010) Fast acceleration of ultimately periodic relations. In: *CAV '10*, LNCS 6174, pp 227–242, doi:[10.1007/978-3-642-14295-6\\_23](https://doi.org/10.1007/978-3-642-14295-6_23)
10. Bozga M, Iosif R, Konečný F (2012) Relational analysis of integer programs. Tech. Rep. TR-2012-10, VERIMAG, URL <https://www-verimag.imag.fr/TR/TR-2012-10.pdf>
11. Bromberger M, Weidenbach C (2016) Fast cube tests for LIA constraint solving. In: *IJCAR '16*, LNCS 9706, pp 116–132, doi:[10.1007/978-3-319-40229-1\\_9](https://doi.org/10.1007/978-3-319-40229-1_9)
12. Chistikov D, Mansutti A, Starchak MR (2024) Integer linear-exponential programming in NP by quantifier elimination. In: *ICALP '24*, LIPIcs 297, pp 132:1–132:20, doi:[10.4230/LIPICS.ICALP.2024.132](https://doi.org/10.4230/LIPICS.ICALP.2024.132)
13. Cimatti A, Griggio A, Schaafsma BJ, Sebastiani R (2013) The MathSAT5 SMT solver. In: *TACAS '13*, LNCS 7795, pp 93–107, doi:[10.1007/978-3-642-36742-7\\_7](https://doi.org/10.1007/978-3-642-36742-7_7)
14. Cimatti A, Griggio A, Irfan A, Roveri M, Sebastiani R (2018) Experimenting on solving nonlinear integer arithmetic with incremental linearization. In: *SAT '18*, LNCS 10929, pp 383–398, doi:[10.1007/978-3-319-94144-8\\_23](https://doi.org/10.1007/978-3-319-94144-8_23)
15. Cimatti A, Griggio A, Irfan A, Roveri M, Sebastiani R (2018) Incremental linearization for satisfiability and verification modulo nonlinear arithmetic and transcendental functions. *ACM Trans Comput Log* 19(3):19:1–19:52, doi:[10.1145/3230639](https://doi.org/10.1145/3230639)
16. Cimatti A, Griggio A, Lipparini E, Sebastiani R (2022) Handling polynomial and transcendental functions in SMT via unconstrained optimisation and topo-

- logical degree test. In: ATVA '22, LNCS 13505, pp 137–153, doi:[10.1007/978-3-031-19992-9\\_9](https://doi.org/10.1007/978-3-031-19992-9_9)
17. Clarke EM, Grumberg O, Jha S, Lu Y, Veith H (2000) Counterexample-guided abstraction refinement. In: CAV '00, LNCS 1855, pp 154–169, doi:[10.1007/10722167\\_15](https://doi.org/10.1007/10722167_15)
  18. Fränzle M, Herde C, Teige T, Ratschan S, Schubert T (2007) Efficient solving of large non-linear arithmetic constraint systems with complex Boolean structure. *J Satisf Boolean Model Comput* 1(3-4):209–236, doi:[10.3233/sat190012](https://doi.org/10.3233/sat190012)
  19. Frohn F, Giesl J (2019) Proving non-termination via loop acceleration. In: FMCAD '19, pp 221–230, doi:[10.23919/FMCAD.2019.8894271](https://doi.org/10.23919/FMCAD.2019.8894271)
  20. Frohn F, Giesl J (2019) Termination of triangular integer loops is decidable. In: CAV '19, LNCS 11562, pp 426–444, doi:[10.1007/978-3-030-25543-5\\_24](https://doi.org/10.1007/978-3-030-25543-5_24)
  21. Frohn F, Naaf M, Brockschmidt M, Giesl J (2020) Inferring lower runtime bounds for integer programs. *ACM Trans Program Lang Syst* 42(3):13:1–13:50, doi:[10.1145/3410331](https://doi.org/10.1145/3410331)
  22. Frohn F, Giesl J (2022) Proving non-termination and lower runtime bounds with LoAT. In: IJCAR '22, LNCS 13385, pp 712–722, doi:[10.1007/978-3-031-10769-6\\_41](https://doi.org/10.1007/978-3-031-10769-6_41)
  23. Frohn F, Giesl J (2023) ADCL: Acceleration Driven Clause Learning for constrained Horn clauses. In: SAS '23, LNCS 14284, pp 259–285, doi:[10.1007/978-3-031-44245-2\\_13](https://doi.org/10.1007/978-3-031-44245-2_13)
  24. Frohn F, Giesl J (2023) Proving non-termination by Acceleration Driven Clause Learning. In: CADE '23, LNCS 14132, pp 220–233, doi:[10.1007/978-3-031-38499-8\\_13](https://doi.org/10.1007/978-3-031-38499-8_13)
  25. Frohn F, Giesl J (2024) Integrating loop acceleration into bounded model checking. In: FM '24, LNCS 14933, pp 73–91, doi:[10.1007/978-3-031-71162-6\\_4](https://doi.org/10.1007/978-3-031-71162-6_4)
  26. Frohn F, Giesl J (2024) Satisfiability modulo exponential integer arithmetic. In: IJCAR '24, LNCS 14739, pp 344–365, doi:[10.1007/978-3-031-63498-7\\_21](https://doi.org/10.1007/978-3-031-63498-7_21)
  27. Frohn F, Giesl J (2024) Evaluation of “Satisfiability modulo exponential integer arithmetic”. URL <https://aprove-developers.github.io/swine-journal-eval>
  28. Frohn F, Giesl J (2026) SwInE website. URL <https://ffrohn.github.io/swine>
  29. Gao S, Avigad J, Clarke EM (2012)  $\delta$ -complete decision procedures for satisfiability over the reals. In: IJCAR '12, LNCS 7364, pp 286–300, doi:[10.1007/978-3-642-31365-3\\_23](https://doi.org/10.1007/978-3-642-31365-3_23)
  30. Giesl J, Rubio A, Sternagel C, Waldmann J, Yamada A (2019) The termination and complexity competition. In: TACAS '19, LNCS 11429, pp 156–166, doi:[10.1007/978-3-030-17502-3\\_10](https://doi.org/10.1007/978-3-030-17502-3_10)
  31. Hark M, Frohn F, Giesl J (2025) Termination of triangular polynomial loops. *Form Methods Syst Des* 65(1):70–132, doi:[10.1007/s10703-023-00440-z](https://doi.org/10.1007/s10703-023-00440-z)
  32. Hurd J (2003) First-order proof tactics in higher-order logic theorem provers. In: STRATA '03, Report NASA/CP-2003-212448, pp 56–68, URL <https://apps.dtic.mil/sti/pdfs/ADA418902.pdf>
  33. Jeannet B, Schrammel P, Sankaranarayanan S (2014) Abstract acceleration of general linear loops. In: POPL '14, pp 529–540, doi:[10.1145/2535838.2535843](https://doi.org/10.1145/2535838.2535843)
  34. Karimov T, Luca F, Nieuwveld J, Ouaknine J, Worrell J (2025) On the decidability of Presburger arithmetic expanded with powers. In: SODA '25, pp 2755–2778, doi:[10.1137/1.9781611978322.89](https://doi.org/10.1137/1.9781611978322.89)

35. Kroening D, Lewis M, Weissenbacher G (2015) Under-approximating loops in C programs for fast counterexample detection. *Formal Methods Syst Des* 47(1):75–92, doi:[10.1007/s10703-015-0228-1](https://doi.org/10.1007/s10703-015-0228-1)
36. Lipparini E, Ratschan S (2023) Satisfiability of non-linear transcendental arithmetic as a certificate search problem. In: *NFM '23*, LNCS 13903, pp 472–488, doi:[10.1007/978-3-031-33170-1\\_29](https://doi.org/10.1007/978-3-031-33170-1_29)
37. Mann M, Wilson A, Zohar Y, Stuntz L, Irfan A, Brown K, Donovan C, Guman A, Tinelli C, Barrett CW (2021) SMT-Switch: A solver-agnostic C++ API for SMT solving. In: *SAT '21*, LNCS 12831, pp 377–386, doi:[10.1007/978-3-030-80223-3\\_26](https://doi.org/10.1007/978-3-030-80223-3_26)
38. de Moura L, Bjørner N (2007) Efficient E-matching for SMT solvers. In: *CADE '07*, LNCS 4603, pp 183–198, doi:[10.1007/978-3-540-73595-3\\_13](https://doi.org/10.1007/978-3-540-73595-3_13)
39. de Moura L, Bjørner N (2008) Z3: An efficient SMT solver. In: *TACAS '08*, LNCS 4963, pp 337–340, doi:[10.1007/978-3-540-78800-3\\_24](https://doi.org/10.1007/978-3-540-78800-3_24)
40. Popescu PG, Díaz-Barrero JL (2006) Certain inequalities for convex functions. *JIPAM* 7(2), URL <http://eudml.org/doc/128518>
41. Pritchard P (1981) A sublinear additive sieve for finding prime numbers. *Commun ACM* 24(1):18–23, doi:[10.1145/358527.358540](https://doi.org/10.1145/358527.358540)
42. Reynolds A, Tinelli C, Goel A, Krstic S, Deters M, Barrett CW (2013) Quantifier instantiation techniques for finite model finding in SMT. In: *CADE '13*, LNCS 7898, pp 377–391, doi:[10.1007/978-3-642-38574-2\\_26](https://doi.org/10.1007/978-3-642-38574-2_26)
43. Reynolds A, Tinelli C, de Moura L (2014) Finding conflicting instances of quantified formulas in SMT. In: *FMCAD '14*, pp 195–202, doi:[10.1109/FMCAD.2014.6987613](https://doi.org/10.1109/FMCAD.2014.6987613)
44. Reynolds A, Deters M, Kuncak V, Tinelli C, Barrett CW (2015) Counterexample-guided quantifier instantiation for synthesis in SMT. In: *CAV '15*, LNCS 9207, pp 198–216, doi:[10.1007/978-3-319-21668-3\\_12](https://doi.org/10.1007/978-3-319-21668-3_12)
45. Reynolds A, Barbosa H, Fontaine P (2018) Revisiting enumerative instantiation. In: *TACAS '18*, LNCS 10806, pp 112–131, doi:[10.1007/978-3-319-89963-3\\_7](https://doi.org/10.1007/978-3-319-89963-3_7)
46. Termination Problems Data Base (TPDB) (2026) URL <http://termination-portal.org/wiki/TPDB>
47. Vasić PM, Pečarić JE (1979) On the Jensen inequality. *Publikacije Elektrotehničkog fakulteta Serija Matematika i fizika* 634/677:50–54, URL <https://www.jstor.org/stable/43668091>
48. Weidenbach C, Dimova D, Fietzke A, Kumar R, Suda M, Wischniewski P (2009) SPASS version 3.5. In: *CADE '09*, LNCS 5663, pp 140–145, doi:[10.1007/978-3-642-02959-2\\_10](https://doi.org/10.1007/978-3-642-02959-2_10)

## A Proofs

### A.1 Proof of Lemma 3.4

**Lemma 3.4** We have

$$\varphi \equiv_{\text{EIA}} \text{FOLDCONSTANTS}(\varphi) \quad \text{and} \quad \varphi \equiv_{\text{EIA}} \text{REWRITE}(\varphi).$$

*Proof.* For constant folding, the claim is trivial, so we only have to show  $\varphi \equiv_{\text{EIA}} \text{REWRITE}(\varphi)$ . To this end, it suffices to show  $\llbracket \ell \rrbracket^{\mathbf{A}} = \llbracket r \rrbracket^{\mathbf{A}}$  for all rewrite rules  $\ell \rightarrow r$  and all  $\mathbf{A} \in \text{EIA}$ . Let  $\mathbf{A} \in \text{EIA}$  be arbitrary but fixed and let  $\llbracket \dots \rrbracket$  denote  $\llbracket \dots \rrbracket^{\mathbf{A}}$ .

For the first rewrite rule  $\exp(x, c) \rightarrow x^c$ , recall that  $x^c$  is syntactic sugar for  $\overbrace{x \cdot \dots \cdot x}^{c \text{ times}}$ , i.e., we have to show  $\llbracket \exp(x, c) \rrbracket = \llbracket \overbrace{x \cdot \dots \cdot x}^{c \text{ times}} \rrbracket$ . We obtain

$$\llbracket \exp(x, c) \rrbracket = \llbracket x \rrbracket^{\llbracket c \rrbracket} = \llbracket x \rrbracket^c = \llbracket \overbrace{x \cdot \dots \cdot x}^{c \text{ times}} \rrbracket = \llbracket \overbrace{x \cdot \dots \cdot x}^{c \text{ times}} \rrbracket,$$

as  $c \in \mathbb{N}$ .

For the second rewrite rule

$$\exp(\exp(x, y), z) \rightarrow \text{ite}(y < 0 \wedge z < 0, \exp(x, -(y \cdot z)), \exp(x, y \cdot z)),$$

first consider the case  $\llbracket y \rrbracket \geq 0$  and  $\llbracket z \rrbracket \geq 0$ . Then we have

$$\begin{aligned} \llbracket \exp(\exp(x, y), z) \rrbracket &= \llbracket \exp(x, y) \rrbracket^{\llbracket z \rrbracket} = \left( \llbracket x \rrbracket^{\llbracket y \rrbracket} \right)^{\llbracket z \rrbracket} = \llbracket x \rrbracket^{\llbracket y \rrbracket \cdot \llbracket z \rrbracket} \\ &= \llbracket x \rrbracket^{\llbracket y \cdot z \rrbracket} = \llbracket \exp(x, y \cdot z) \rrbracket \\ &= \llbracket \text{ite}(y < 0 \wedge z < 0, \exp(x, -(y \cdot z)), \exp(x, y \cdot z)) \rrbracket. \end{aligned}$$

Next, consider the case  $\llbracket x \rrbracket^{\llbracket y \rrbracket} \notin \mathbb{Z}$ , where we consider the cases  $\llbracket z \rrbracket > 0$ ,  $\llbracket z \rrbracket = 0$ , and  $\llbracket z \rrbracket < 0$  independently. If  $\llbracket z \rrbracket > 0$ , then we have

$$\begin{aligned} \llbracket \exp(\exp(x, y), z) \rrbracket &= \llbracket \exp(\llbracket \exp(x, y) \rrbracket, z) \rrbracket = \llbracket \exp(0, z) \rrbracket = 0^{\llbracket z \rrbracket} \\ &= 0 = \llbracket \exp(x, y \cdot z) \rrbracket \\ &= \llbracket \text{ite}(y < 0 \wedge z < 0, \exp(x, -(y \cdot z)), \exp(x, y \cdot z)) \rrbracket, \end{aligned}$$

where the but-last equality holds as  $\llbracket x \rrbracket^{\llbracket y \rrbracket} \notin \mathbb{Z}$  implies  $\llbracket x \rrbracket^{\llbracket y \rrbracket \cdot c} \notin \mathbb{Z}$  for all  $c \in \mathbb{N}_{>0}$ . If  $\llbracket z \rrbracket = 0$ , we have

$$\begin{aligned} \llbracket \exp(\exp(x, y), z) \rrbracket &= \llbracket \exp(\llbracket \exp(x, y) \rrbracket, z) \rrbracket = \llbracket \exp(0, z) \rrbracket = 0^{\llbracket z \rrbracket} \\ &= 0^0 = 1 = \llbracket x \rrbracket^0 \\ &= \llbracket x \rrbracket^{\llbracket y \rrbracket \cdot \llbracket z \rrbracket} = \llbracket x \rrbracket^{\llbracket y \cdot z \rrbracket} = \llbracket \exp(x, y \cdot z) \rrbracket \\ &= \llbracket \text{ite}(y < 0 \wedge z < 0, \exp(x, -(y \cdot z)), \exp(x, y \cdot z)) \rrbracket. \end{aligned}$$

If  $\llbracket z \rrbracket < 0$ , we have

$$\begin{aligned} \llbracket \exp(\exp(x, y), z) \rrbracket &= \llbracket \exp(\llbracket \exp(x, y) \rrbracket, z) \rrbracket = \llbracket \exp(0, z) \rrbracket = 0 = \llbracket \exp(x, -(y \cdot z)) \rrbracket \\ &= \llbracket \text{ite}(y < 0 \wedge z < 0, \exp(x, -(y \cdot z)), \exp(x, y \cdot z)) \rrbracket, \end{aligned}$$

where the but-last equality holds as  $\llbracket x \rrbracket^{\llbracket y \rrbracket} \notin \mathbb{Z}$  implies  $\llbracket y \rrbracket < 0$ , which, together with  $\llbracket z \rrbracket < 0$ , implies  $\llbracket -(y \cdot z) \rrbracket < 0$ . Moreover,  $\llbracket x \rrbracket^{\llbracket y \rrbracket} \notin \mathbb{Z}$  implies  $|\llbracket x \rrbracket| \neq 1$  which, together with  $\llbracket -(y \cdot z) \rrbracket < 0$ , implies  $\llbracket x \rrbracket^{\llbracket -(y \cdot z) \rrbracket} \notin \mathbb{Z}$ , i.e.,  $\llbracket \exp(x, -(y \cdot z)) \rrbracket = 0$ .

Now the only missing case is  $\llbracket y \rrbracket < 0$  and  $\llbracket x \rrbracket^{\llbracket y \rrbracket} \in \mathbb{Z}$ , which implies  $|\llbracket x \rrbracket| = 1 = |\llbracket x \rrbracket^{\llbracket y \rrbracket}|$ . Then we have

$$\begin{aligned} \llbracket \exp(\exp(x, y), z) \rrbracket &= \llbracket \exp(\llbracket \exp(x, y) \rrbracket, z) \rrbracket = \llbracket \exp(\llbracket x \rrbracket^{\llbracket y \rrbracket}, z) \rrbracket = \left( \llbracket x \rrbracket^{\llbracket y \rrbracket} \right)^{\llbracket z \rrbracket} \\ &= \llbracket x \rrbracket^{\llbracket y \rrbracket \cdot \llbracket z \rrbracket} = \llbracket x \rrbracket^{\llbracket y \cdot z \rrbracket} = \llbracket \exp(x, \pm(y \cdot z)) \rrbracket \\ &= \llbracket \text{ite}(y < 0 \wedge z < 0, \exp(x, -(y \cdot z)), \exp(x, y \cdot z)) \rrbracket, \end{aligned}$$

where the but-last equality holds for both  $\llbracket \exp(x, y \cdot z) \rrbracket$  and  $\llbracket \exp(x, -(y \cdot z)) \rrbracket$ , as we have  $c^d = c^{-d}$  if  $|c| = 1$ .

For the third rewrite rule

$$\exp(x, y) \cdot \exp(x, z) \rightarrow \text{ite}(\text{abs}(x) \neq 1 \wedge (y < 0 \vee z < 0), 0, \exp(x, y + z)),$$

first assume  $\llbracket \text{abs}(x) \rrbracket = 1 \vee (\llbracket y \rrbracket \geq 0 \wedge \llbracket z \rrbracket \geq 0)$ . Then we have  $\llbracket x \rrbracket^{\llbracket y \rrbracket}, \llbracket x \rrbracket^{\llbracket z \rrbracket}, \llbracket x \rrbracket^{\llbracket y+z \rrbracket} \in \mathbb{Z}$  and we need to show

$$\llbracket \exp(x, y) \rrbracket \cdot \llbracket \exp(x, z) \rrbracket = \llbracket \exp(x, y + z) \rrbracket.$$

We get:

$$\llbracket \exp(x, y) \cdot \exp(x, z) \rrbracket = \llbracket x \rrbracket^{\llbracket y \rrbracket} \cdot \llbracket x \rrbracket^{\llbracket z \rrbracket} = \llbracket x \rrbracket^{\llbracket y+z \rrbracket} = \llbracket \exp(x, y+z) \rrbracket$$

Otherwise, we have  $\llbracket x \rrbracket^{\llbracket y \rrbracket} \notin \mathbb{Z}$  or  $\llbracket x \rrbracket^{\llbracket z \rrbracket} \notin \mathbb{Z}$  and we need to show  $\llbracket \exp(x, y) \cdot \exp(x, z) \rrbracket = 0$ . If  $\llbracket x \rrbracket^{\llbracket y \rrbracket} \notin \mathbb{Z}$ , we get

$$\llbracket \exp(x, y) \cdot \exp(x, z) \rrbracket = 0 \cdot \llbracket \exp(x, z) \rrbracket = 0$$

and the case  $\llbracket x \rrbracket^{\llbracket z \rrbracket} \notin \mathbb{Z}$  is analogous.

For the fourth rewrite rule  $\exp(x, y) \cdot \exp(z, y) \rightarrow \exp(x \cdot z, y)$ , first consider the case  $\llbracket x \rrbracket^{\llbracket y \rrbracket}, \llbracket z \rrbracket^{\llbracket y \rrbracket} \in \mathbb{Z}$ . Then we have

$$\begin{aligned} \llbracket \exp(x, y) \cdot \exp(z, y) \rrbracket &= \llbracket \exp(x, y) \rrbracket \cdot \llbracket \exp(z, y) \rrbracket = \llbracket x \rrbracket^{\llbracket y \rrbracket} \cdot \llbracket z \rrbracket^{\llbracket y \rrbracket} = (\llbracket x \rrbracket \cdot \llbracket z \rrbracket)^{\llbracket y \rrbracket} \\ &= \llbracket x \cdot z \rrbracket^{\llbracket y \rrbracket} = \llbracket \exp(x \cdot z, y) \rrbracket. \end{aligned}$$

If  $\llbracket x \rrbracket^{\llbracket y \rrbracket} \notin \mathbb{Z}$ , then we have  $\llbracket y \rrbracket < 0$ . Thus,  $\llbracket x \rrbracket^{\llbracket y \rrbracket} \in (-1, 0) \cup (0, 1)$  and  $\llbracket z \rrbracket^{\llbracket y \rrbracket} \in \{-1, 1\}$  (if  $\llbracket z \rrbracket \in \{-1, 1\}$ ), or  $\llbracket z \rrbracket^{\llbracket y \rrbracket} \in (-1, 0) \cup (0, 1)$ . Therefore,  $\llbracket x \rrbracket^{\llbracket y \rrbracket} \cdot \llbracket z \rrbracket^{\llbracket y \rrbracket} \notin \mathbb{Z}$ . Thus, we have

$$\llbracket \exp(x, y) \cdot \exp(z, y) \rrbracket = 0 = \llbracket \exp(x \cdot z, y) \rrbracket.$$

The case  $\llbracket x \rrbracket^{\llbracket z \rrbracket} \notin \mathbb{Z}$  is analogous.  $\square$

## A.2 Proof of Lemma 3.8

**Lemma 3.8** Let  $s, t$  be terms of sort **Int**. Then  $\text{NEG}_1 - \text{NEG}_3$  are EIA-valid.

*Proof.* Let  $\mathbf{A} \in \text{EIA}$  again be arbitrary but fixed and let  $\llbracket \dots \rrbracket$  denote  $\llbracket \dots \rrbracket^{\mathbf{A}}$ . For  $\text{NEG}_1$ , assume  $\llbracket \text{divisible}_2(t) \rrbracket = \mathbf{true}$ , i.e., assume that  $\llbracket t \rrbracket$  is even. Then it remains to show

$$\llbracket \exp(s, t) = \exp(-s, t) \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket \exp(s, t) \rrbracket = \llbracket \exp(-s, t) \rrbracket.$$

First consider the case  $\llbracket s \rrbracket^{\llbracket t \rrbracket} \in \mathbb{Z}$ . Then we have:

$$\begin{aligned} \llbracket \exp(s, t) \rrbracket &= \llbracket s \rrbracket^{\llbracket t \rrbracket} \\ &= \llbracket -s \rrbracket^{\llbracket t \rrbracket} && (\text{as } \llbracket t \rrbracket \text{ is even}) \\ &= \llbracket \exp(-s, t) \rrbracket \end{aligned}$$

If  $\llbracket s \rrbracket^{\llbracket t \rrbracket} \notin \mathbb{Z}$ , then  $\llbracket -s \rrbracket^{\llbracket t \rrbracket} \notin \mathbb{Z}$ , so we have

$$\llbracket \exp(s, t) \rrbracket = 0 = \llbracket \exp(-s, t) \rrbracket.$$

For  $\text{NEG}_2$ , assume  $\llbracket \text{divisible}_2(t) \rrbracket = \mathbf{false}$ , i.e., assume that  $\llbracket t \rrbracket$  is odd. Then it remains to show

$$\llbracket \exp(s, t) = -\exp(-s, t) \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket \exp(s, t) \rrbracket = \llbracket -\exp(-s, t) \rrbracket.$$

First consider the case  $\llbracket s \rrbracket^{\llbracket t \rrbracket} \in \mathbb{Z}$ . Then we have:

$$\begin{aligned} \llbracket \exp(s, t) \rrbracket &= \llbracket s \rrbracket^{\llbracket t \rrbracket} \\ &= -\llbracket -s \rrbracket^{\llbracket t \rrbracket} && (\text{as } \llbracket t \rrbracket \text{ is odd}) \\ &= -\llbracket \exp(-s, t) \rrbracket \\ &= \llbracket -\exp(-s, t) \rrbracket \end{aligned}$$

If  $\llbracket s \rrbracket^{\llbracket t \rrbracket} \notin \mathbb{Z}$ , then  $\llbracket -s \rrbracket^{\llbracket t \rrbracket} \notin \mathbb{Z}$ , so we have

$$\llbracket \exp(s, t) \rrbracket = 0 = \llbracket -\exp(-s, t) \rrbracket.$$

For  $\text{NEG}_3$ , assume  $\llbracket \text{abs}(s) \neq 1 \wedge t < 0 \rrbracket = \mathbf{true}$ , i.e., assume  $|\llbracket s \rrbracket| \neq 1$  and  $\llbracket t \rrbracket < 0$ . Then it remains to show

$$\llbracket \exp(s, t) \rrbracket = 0$$

which follows as we have  $\llbracket s \rrbracket^{\llbracket t \rrbracket} \notin \mathbb{Z}$ .  $\square$

## A.3 Proof of Lemma 3.10

**Lemma 3.10** Let  $s_1, s_2, t_1, t_2$  be terms of sort **Int**. Then **MON** is EIA-valid.

*Proof.* Let  $\mathbf{A} \in \text{EIA}$  be arbitrary but fixed and let  $\llbracket \dots \rrbracket$  denote  $\llbracket \dots \rrbracket^{\mathbf{A}}$ . Assume

$$\llbracket s_2 \geq s_1 \geq 2 \wedge t_2 \geq t_1 > 0 \wedge (s_2 > s_1 \vee t_2 > t_1) \rrbracket = \mathbf{true},$$

i.e.,

$$\llbracket s_2 \rrbracket \geq \llbracket s_1 \rrbracket \geq 2, \llbracket t_2 \rrbracket \geq \llbracket t_1 \rrbracket > 0, \text{ and } (\llbracket s_2 \rrbracket > \llbracket s_1 \rrbracket \text{ or } \llbracket t_2 \rrbracket > \llbracket t_1 \rrbracket).$$

Then it remains to prove

$$\llbracket \exp(s_2, t_2) > \exp(s_1, t_1) \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket \exp(s_2, t_2) \rrbracket > \llbracket \exp(s_1, t_1) \rrbracket.$$

First note that  $\lambda x. x^{\llbracket t_1 \rrbracket}$  is strictly monotonically increasing on  $\mathbb{N}_{\geq 2}$  as  $\llbracket t_1 \rrbracket > 0$ , and  $\lambda y. \llbracket s_1 \rrbracket^y$  is strictly monotonically increasing on  $\mathbb{N}_{>0}$  as  $\llbracket s_1 \rrbracket \geq 2$ . Since we have  $\llbracket s_2 \rrbracket, \llbracket s_1 \rrbracket \in \mathbb{N}_{\geq 2}$  and  $\llbracket t_2 \rrbracket, \llbracket t_1 \rrbracket \in \mathbb{N}_{>0}$ , we get

$$\llbracket \exp(s_2, t_2) \rrbracket = \llbracket s_2 \rrbracket^{\llbracket t_2 \rrbracket} > \llbracket s_1 \rrbracket^{\llbracket t_1 \rrbracket} = \llbracket \exp(s_1, t_1) \rrbracket$$

due to monotonicity, as we have  $\llbracket s_2 \rrbracket \geq \llbracket s_1 \rrbracket$  and  $\llbracket t_2 \rrbracket \geq \llbracket t_1 \rrbracket$ , where at least one of both inequations is strict.  $\square$

## A.4 Proof of Lemma 3.12

**Lemma 3.12** Let  $s, t$  be terms of sort **Int**. Then **BND<sub>1</sub>** – **BND<sub>5</sub>** are EIA-valid.

*Proof.* Let  $\mathbf{A} \in \text{EIA}$  again be arbitrary but fixed and let  $\llbracket \dots \rrbracket$  denote  $\llbracket \dots \rrbracket^{\mathbf{A}}$ . For **BND<sub>1</sub>**, assume

$$\llbracket t = 0 \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket t \rrbracket = 0.$$

Then it remains to show

$$\llbracket \exp(s, t) = 1 \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket \exp(s, t) \rrbracket = 1.$$

We have

$$\llbracket \exp(s, t) \rrbracket = \llbracket \exp(s, 0) \rrbracket = \llbracket s \rrbracket^0 = 1.$$

For **BND<sub>2</sub>**, assume

$$\llbracket t = 1 \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket t \rrbracket = 1.$$

Then it remains to show

$$\llbracket \exp(s, t) = s \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket \exp(s, t) \rrbracket = \llbracket s \rrbracket.$$

We have

$$\llbracket \exp(s, t) \rrbracket = \llbracket \exp(s, 1) \rrbracket = \llbracket s \rrbracket^1 = \llbracket s \rrbracket.$$

For **BND<sub>3</sub>**, assume

$$\llbracket s = 0 \wedge t \neq 0 \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket s \rrbracket = 0 \text{ and } \llbracket t \rrbracket \neq 0.$$

Then it remains to show

$$\llbracket \exp(s, t) = 0 \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket \exp(s, t) \rrbracket = 0.$$

We have

$$\llbracket \exp(s, t) \rrbracket = \llbracket \exp(0, t) \rrbracket = 0$$

where the last equality holds as we have  $\llbracket t \rrbracket \neq 0$ , which implies  $\llbracket \exp(0, t) \rrbracket = 0$  by the definition of EIA.

For **BND<sub>4</sub>**, assume

$$\llbracket s = 1 \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket s \rrbracket = 1.$$

Then it remains to show

$$\llbracket \text{exp}(s, t) = 1 \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket \text{exp}(s, t) \rrbracket = 1.$$

We have

$$\llbracket \text{exp}(s, t) \rrbracket = \llbracket \text{exp}(1, t) \rrbracket = 1^{\llbracket t \rrbracket} = 1.$$

For [BND<sub>5</sub>](#), assume

$$\llbracket s + t \geq 5 \wedge s \geq 2 \wedge t \geq 2 \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket s \rrbracket + \llbracket t \rrbracket \geq 5, \llbracket s \rrbracket \geq 2, \text{ and } \llbracket t \rrbracket \geq 2.$$

Then it remains to show

$$\llbracket \text{exp}(s, t) > s \cdot t + 1 \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket \text{exp}(s, t) \rrbracket > \llbracket s \rrbracket \cdot \llbracket t \rrbracket + 1.$$

We use induction on  $\llbracket s \rrbracket + \llbracket t \rrbracket$ . In the base case, we have either  $\llbracket s \rrbracket = 2$  and  $\llbracket t \rrbracket = 3$ , or  $\llbracket s \rrbracket = 3$  and  $\llbracket t \rrbracket = 2$ . In the former case, we have:

$$\llbracket \text{exp}(s, t) \rrbracket = \llbracket s \rrbracket^{\llbracket t \rrbracket} = 2^3 = 8 > 7 = 2 \cdot 3 + 1 = \llbracket s \rrbracket \cdot \llbracket t \rrbracket + 1$$

In the latter case, we have:

$$\llbracket \text{exp}(s, t) \rrbracket = \llbracket s \rrbracket^{\llbracket t \rrbracket} = 3^2 = 9 > 7 = 3 \cdot 2 + 1 = \llbracket s \rrbracket \cdot \llbracket t \rrbracket + 1$$

For the induction step, assume  $\llbracket s \rrbracket + \llbracket t \rrbracket \geq 6$ . If  $\llbracket t \rrbracket \geq 3$ , then:

$$\begin{aligned} \llbracket \text{exp}(s, t) \rrbracket &= \llbracket s \rrbracket^{\llbracket t \rrbracket} \\ &= \llbracket s \rrbracket^{\llbracket t \rrbracket - 1} \cdot \llbracket s \rrbracket \\ &> (\llbracket s \rrbracket \cdot (\llbracket t \rrbracket - 1) + 1) \cdot \llbracket s \rrbracket && \text{(by the induction hypothesis)} \\ &\geq (\llbracket s \rrbracket \cdot (\llbracket t \rrbracket - 1) + 1) \cdot 2 && \text{(as } \llbracket s \rrbracket \geq 2) \\ &= (\llbracket s \rrbracket \cdot (\llbracket t \rrbracket - 1) + 1) + (\llbracket s \rrbracket \cdot (\llbracket t \rrbracket - 1) + 1) \\ &\geq (\llbracket s \rrbracket \cdot (\llbracket t \rrbracket - 1) + 1) + (\llbracket s \rrbracket \cdot 2 + 1) && \text{(as } \llbracket t \rrbracket \geq 3) \\ &> (\llbracket s \rrbracket \cdot (\llbracket t \rrbracket - 1) + 1) + \llbracket s \rrbracket \\ &= \llbracket s \rrbracket \cdot \llbracket t \rrbracket + 1 \end{aligned}$$

If  $\llbracket t \rrbracket = 2$ , then  $\llbracket s \rrbracket \geq 4$ . Now we have

$$\begin{aligned} \llbracket \text{exp}(s, t) \rrbracket &= \llbracket s \rrbracket^{\llbracket t \rrbracket} \\ &= \llbracket s \rrbracket^2 \\ &= (\llbracket s - 1 \rrbracket + 1)^2 \\ &= \llbracket s - 1 \rrbracket^2 + 2 \cdot \llbracket s - 1 \rrbracket + 1 \\ &> \llbracket s - 1 \rrbracket \cdot 2 + 1 + 2 \cdot \llbracket s - 1 \rrbracket + 1 && \text{(by the induction hypothesis)} \\ &\geq \llbracket s - 1 \rrbracket \cdot 2 + 8 && \text{(as } \llbracket s \rrbracket \geq 4 \text{ and thus, } \llbracket s - 1 \rrbracket \geq 3) \\ &= \llbracket s \rrbracket \cdot 2 + 6 \\ &= \llbracket s \rrbracket \cdot \llbracket t \rrbracket + 6 \\ &> \llbracket s \rrbracket \cdot \llbracket t \rrbracket + 1 \end{aligned}$$

□

## A.5 Proof of Lemma [3.14](#)

**Lemma 3.14** Let  $s, t$  be terms of sort **Int** and let  $d \in \mathbb{N}$  be prime. Then [PRIME](#) is EIA-valid.

*Proof.* Again, let  $\mathbf{A} \in \text{EIA}$  be arbitrary but fixed and let  $\llbracket \dots \rrbracket$  denote  $\llbracket \dots \rrbracket^{\mathbf{A}}$ . Assume

$$\llbracket t > 0 \rrbracket = \mathbf{true}, \quad \text{i.e.,} \quad \llbracket t \rrbracket > 0.$$

We need to prove

$$\llbracket \text{divisible}_d(\exp(s, t)) \rrbracket \iff \llbracket \text{divisible}_d(s) \rrbracket = \mathbf{true},$$

i.e.,

$$\llbracket \text{divisible}_d(\exp(s, t)) \rrbracket \quad \text{iff} \quad \llbracket \text{divisible}_d(s) \rrbracket.$$

So we have to prove that  $\llbracket s \rrbracket$  and  $\llbracket s \rrbracket^{\llbracket t \rrbracket}$  have the same prime factors. If  $P$  is the multiset of  $\llbracket s \rrbracket$ 's prime factors, i.e.,  $\prod P = \llbracket s \rrbracket$ , then  $\prod_{i=1}^{\llbracket t \rrbracket} P = \llbracket s \rrbracket^{\llbracket t \rrbracket}$ , so the claim follows.  $\square$

### A.6 Proof of Lemma 3.16

**Lemma 3.16** Let  $s_1, s_2, t_1, t_2$  be terms of sort **Int** and let  $d \in \mathbb{N}_{>0}$ . Then **IND** is EIA-valid.

*Proof.* Let  $\mathbf{A} \in \text{EIA}$  be an arbitrary but fixed model of

$$s_1 = s_2 \wedge t_2 - d = t_1 \geq 0$$

and let  $\llbracket \dots \rrbracket$  denote  $\llbracket \dots \rrbracket^{\mathbf{A}}$ . Then we have:

$$\begin{aligned} \llbracket \exp(s_1, t_1) \cdot s_1^d \rrbracket &= \llbracket \exp(s_1, t_1) \rrbracket \cdot \llbracket s_1 \rrbracket^d \\ &= \llbracket s_1 \rrbracket^{\llbracket t_1 \rrbracket} \cdot \llbracket s_1 \rrbracket^d && (\text{as } \mathbf{A} \models t_1 \geq 0) \\ &= \llbracket s_1 \rrbracket^{\llbracket t_1 \rrbracket + d} \\ &= \llbracket s_1 \rrbracket^{\llbracket t_2 \rrbracket} && (\text{as } \mathbf{A} \models t_2 - d = t_1) \\ &= \llbracket s_2 \rrbracket^{\llbracket t_2 \rrbracket} && (\text{as } \mathbf{A} \models s_1 = s_2) \\ &= \llbracket \exp(s_2, t_2) \rrbracket && (\text{as } \mathbf{A} \models t_2 > 0) \end{aligned}$$

$\square$

### A.7 Proof of Lemma 3.17

**Lemma 3.17** Let  $c \geq 0$ ,  $w_1, w_2 \in \mathbb{R}_{>c}$ ,  $w_1 < w_2$ , and let  $f : \mathbb{R}_{>c} \rightarrow \mathbb{R}_{>0}$  be convex. Then

$$\begin{aligned} \forall x \in [w_1, w_2]. \quad f(x) &\leq f(w_1) + \frac{f(w_2) - f(w_1)}{w_2 - w_1} \cdot (x - w_1) && \text{and} \\ \forall x \in \mathbb{R}_{>c} \setminus (w_1, w_2). \quad f(x) &\geq f(w_1) + \frac{f(w_2) - f(w_1)}{w_2 - w_1} \cdot (x - w_1). \end{aligned}$$

*Proof.* For the first inequation, recall that a function  $f : \mathbb{R}_{>c} \rightarrow \mathbb{R}_{>0}$  is convex if for all  $w_1, w_2 \in \mathbb{R}_{>c}$  and all  $v \in [0, 1]$ , we have

$$f(v \cdot w_2 + (1 - v) \cdot w_1) \leq v \cdot f(w_2) + (1 - v) \cdot f(w_1). \quad (\text{A.1})$$

Let  $x \in [w_1, w_2]$  and  $v = \frac{x - w_1}{w_2 - w_1}$ . Then we have  $v \in [0, 1]$  and

$$\begin{aligned} f(x) &= f(v \cdot w_2 + (1 - v) \cdot w_1) \\ &\leq v \cdot f(w_2) + (1 - v) \cdot f(w_1) && (\text{by convexity of } f \text{ and } v \in [0, 1]) \\ &= f(w_1) + \frac{f(w_2) - f(w_1)}{w_2 - w_1} \cdot (x - w_1), \end{aligned}$$

as desired.

For the second inequation, note that dually to (A.1), convex functions  $f : \mathbb{R}_{>c} \rightarrow \mathbb{R}_{>0}$  satisfy

$$f(v \cdot w_2 + (1-v) \cdot w_1) \geq v \cdot f(w_2) + (1-v) \cdot f(w_1)$$

for all  $w_1, w_2 \in \mathbb{R}_{>c}$  and  $v \notin (0, 1)$  such that  $v \cdot w_2 + (1-v) \cdot w_1 \in \mathbb{R}_{>c}$  [40, 47]. Let  $x \in \mathbb{R}_{>c} \setminus (w_1, w_2)$  and  $v = \frac{x-w_1}{w_2-w_1}$ . Then we have  $v \notin (0, 1)$  and

$$\begin{aligned} f(x) &= f(v \cdot w_2 + (1-v) \cdot w_1) \\ &\geq v \cdot f(w_2) + (1-v) \cdot f(w_1) \quad (\text{by convexity of } f \text{ and } v \notin (0, 1)) \\ &= f(w_1) + \frac{f(w_2) - f(w_1)}{w_2 - w_1} \cdot (x - w_1), \end{aligned}$$

as desired.  $\square$

### A.8 Proof of Lemma 3.19

**Lemma 3.19** Let  $c^+ \geq c^- > 0$  and  $d^+ \geq d^- > 0$ . Then  $\text{IP}_{\text{up}}$  is EIA-valid.

*Proof.* Again, let  $\mathbf{A} \in \text{EIA}$  be arbitrary but fixed and let  $\llbracket \dots \rrbracket$  denote  $\llbracket \dots \rrbracket^{\mathbf{A}}$ . Assume  $\llbracket s \rrbracket \in [c^\pm]$  and  $\llbracket t \rrbracket \in [d^\pm]$ . Then we need to show

$$\llbracket \text{exp}(s, t) \leq \text{ip}^{[c^\pm][d^\pm]}(s, t) \rrbracket = \text{true},$$

i.e.,

$$\llbracket \text{exp}(s, t) \rrbracket \leq \llbracket \text{ip}^{[c^\pm][d^\pm]}(s, t) \rrbracket.$$

By (3.15), we have

$$\llbracket s \rrbracket^{\llbracket t \rrbracket} \leq \llbracket \text{ip}^{[c^\pm][d^\pm]}(s, t) \rrbracket.$$

As  $\llbracket t \rrbracket \in [d^\pm]$  and  $d^- > 0$  imply  $\llbracket t \rrbracket > 0$ , we have  $\llbracket \text{exp}(s, t) \rrbracket = \llbracket s \rrbracket^{\llbracket t \rrbracket}$ , and thus the claim follows.  $\square$

### A.9 Proof of Lemma 3.21

**Lemma 3.21** Let  $c, d \in \mathbb{N}_{>0}$ . Then  $\text{IP}_{\text{low}}$  is EIA-valid.

*Proof.* Again, let  $\mathbf{A} \in \text{EIA}$  be arbitrary but fixed and let  $\llbracket \dots \rrbracket$  denote  $\llbracket \dots \rrbracket^{\mathbf{A}}$ . Assume  $\llbracket s \rrbracket > 0$  and  $\llbracket t \rrbracket \geq d$ . Then we need to show

$$\llbracket \text{exp}(s, t) \geq \text{ip}^{[c..c+1][d..d+1]}(s, t) \rrbracket = \text{true},$$

i.e.,

$$\llbracket \text{exp}(s, t) \rrbracket \geq \llbracket \text{ip}^{[c..c+1][d..d+1]}(s, t) \rrbracket.$$

By (3.19), we have

$$\llbracket s \rrbracket^{\llbracket t \rrbracket} \geq \llbracket \text{ip}^{[c..c+1][d..d+1]}(s, t) \rrbracket.$$

As  $\llbracket t \rrbracket \geq d$  and  $d > 0$  imply  $\llbracket t \rrbracket > 0$ , we have  $\llbracket \text{exp}(s, t) \rrbracket = \llbracket s \rrbracket^{\llbracket t \rrbracket}$ , and thus the claim follows.  $\square$

### A.10 Proof of Thm. 3.25

**Theorem 3.25 (Soundness of Alg. 2)** If Alg. 2 returns **sat**, then  $\varphi$  is satisfiable in EIA. If Alg. 2 returns **unsat**, then  $\varphi$  is unsatisfiable in EIA.

*Proof.* Immediate consequence of Lemmas 3.4, 3.8, 3.10, 3.12, 3.14, 3.16, 3.19, and 3.21.  $\square$

## A.11 Proof of Thm. 3.26

**Theorem 3.26 (Progress Theorem)** If  $\mathbf{A}$  is a spurious counterexample and  $\mathcal{L}$  is computed as in Alg. 2, then

$$\mathbf{A} \not\models \bigwedge \mathcal{L}.$$

*Proof.* Let  $\mathbf{A} \in \text{EIA}$  be arbitrary but fixed and let  $\llbracket \dots \rrbracket$  denote  $\llbracket \dots \rrbracket^{\mathbf{A}}$ . Since  $\mathbf{A}$  is a spurious counterexample, there is a subexpression  $\exp(s, t)$  of  $\varphi$  such that  $\llbracket \exp(s, t) \rrbracket \neq \llbracket \exp(s, t) \rrbracket^{\text{EIA}}$ . We now perform a case analysis to show that  $\mathbf{A}$  violates at least one of our lemmas.

*Case 1 – Positive Arguments:* First assume  $\llbracket s \rrbracket, \llbracket t \rrbracket \in \mathbb{N}_{>0}$ . Then

- $\llbracket \exp(s, t) \rrbracket > \llbracket \exp(s, t) \rrbracket^{\text{EIA}}$  implies  $\llbracket \text{IP}_{\text{up}} \rrbracket = \text{false}$  (as remarked after  $\text{IP}_{\text{up}}$ ), and
- $\llbracket \exp(s, t) \rrbracket < \llbracket \exp(s, t) \rrbracket^{\text{EIA}}$  implies  $\llbracket \text{IP}_{\text{low}} \rrbracket = \text{false}$  (as remarked after  $\text{IP}_{\text{low}}$ ).

*Case 2 – Zero as Argument:* Next, consider the case  $\llbracket s \rrbracket = 0$  or  $\llbracket t \rrbracket = 0$ . Then

- $t = 0$  implies  $\llbracket \text{BND}_1 \rrbracket = \text{false}$  and
- $s = 0 \wedge t \neq 0$  implies  $\llbracket \text{BND}_3 \rrbracket = \text{false}$ .

*Case 3 – Negative Second Argument:* Next, assume  $\llbracket t \rrbracket < 0$  and  $\llbracket s \rrbracket \neq 0$  (as  $\llbracket s \rrbracket = 0$  has already been handled in Case 2).

- If  $\llbracket s \rrbracket = 1$ , then  $\llbracket \exp(s, t) \rrbracket \neq \llbracket \exp(s, t) \rrbracket^{\text{EIA}} = 1$  implies  $\llbracket \text{BND}_4 \rrbracket = \text{false}$ .
- If  $\llbracket s \rrbracket = -1$ , then
  - If  $\llbracket t \rrbracket$  is even, then either  $\llbracket \text{NEG}_1 \rrbracket = \text{false}$  or  $\llbracket \exp(s, t) \rrbracket = \llbracket \exp(-s, t) \rrbracket$ . But then

$$\llbracket \exp(-s, t) \rrbracket = \llbracket \exp(s, t) \rrbracket \neq \llbracket \exp(s, t) \rrbracket^{\text{EIA}} = \llbracket \exp(-s, t) \rrbracket^{\text{EIA}} = 1$$

implies  $\llbracket \text{BND}_4 \rrbracket = \text{false}$ .

- if  $\llbracket t \rrbracket$  is odd, then either  $\llbracket \text{NEG}_2 \rrbracket = \text{false}$  or  $\llbracket \exp(s, t) \rrbracket = \llbracket -\exp(-s, t) \rrbracket$ . But then

$$\llbracket -\exp(-s, t) \rrbracket = \llbracket \exp(s, t) \rrbracket \neq \llbracket \exp(s, t) \rrbracket^{\text{EIA}} = \llbracket -\exp(-s, t) \rrbracket^{\text{EIA}} = -1$$

implies  $\llbracket \text{BND}_4 \rrbracket = \text{false}$ .

- If  $\llbracket s \rrbracket \notin \{-1, 1\}$ , then  $\llbracket \text{NEG}_3 \rrbracket = \text{false}$ .

*Case 4 – Negative First Argument:* Now assume  $\llbracket s \rrbracket < 0$  and  $\llbracket t \rrbracket > 0$  (as  $\llbracket t \rrbracket \leq 0$  has already been handled in Case 2 and 3). Since  $\exp(-s, t)$  is relevant, the set  $\mathcal{L}$  contains bounding and interpolation lemmas for  $\exp(-s, t)$ . Hence, if  $\llbracket \exp(-s, t) \rrbracket \neq \llbracket \exp(-s, t) \rrbracket^{\text{EIA}}$ , then one of these lemmas is violated by  $\mathbf{A}$ , as argued above. Thus, assume  $\llbracket \exp(-s, t) \rrbracket = \llbracket \exp(-s, t) \rrbracket^{\text{EIA}}$ . Then

- if  $\llbracket t \rrbracket$  is even, then  $\llbracket \text{NEG}_1 \rrbracket = \text{false}$ , and
- if  $\llbracket t \rrbracket$  is odd, then  $\llbracket \text{NEG}_2 \rrbracket = \text{false}$ .

□